

LLM Server Software

Performing Web and Internet Services via LLM

Registry: [\[HOWL-VDR-25-2026\]](#)

Series Path: [\[HOWL-VDR-1-2026\]](#) → [\[HOWL-VDR-2-2026\]](#) → [\[HOWL-MATH-3-2026\]](#) → [\[HOWL-MATH-4-2026\]](#) → ... → [\[HOWL-VDR-14-2026\]](#) → ... → [\[HOWL-VDR-21-2026\]](#) → [\[HOWL-VDR-22-2026\]](#) → [\[HOWL-VDR-23-2026\]](#) → [\[HOWL-VDR-24-2026\]](#) → [\[HOWL-VDR-25-2026\]](#)

DOI: 10.5281/zenodo.20258674

Date: May 2026

Domain: Computer Architecture / Adaptive Precision Arithmetic

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Opus 4.6.

Abstract

This paper demonstrates that web and internet services — HTTP servers, email systems, chat protocols, authentication services, database interfaces, file storage, monitoring, streaming, and IoT infrastructure — can be implemented as LM Software applications within the VDR-LLM-Prolog architecture [\[HOWL-VDR-14-2026\]](#). Each service is a configured language model session that speaks a protocol through grammar templates, processes requests through Prolog rules over exact arithmetic primitives, and maintains state in a hierarchical knowledge base tree. The language model provides judgment only when requests require interpretation beyond stored rules. Protocol compliance is structural — grammars enforce correct framing, field ordering, and encoding. Security is structural — grants and scope determine what each connection can access. Audit is automatic — every operation is logged with full provenance.

This extends the LM Software concept [\[HOWL-VDR-24-2026\]](#) from application-level programs to infrastructure-level services. The same development lifecycle applies: interactive configuration, testing, snapshotting, clone-per-connection deployment, and improvement through accumulated rules. The result is server software developed through conversation rather than compiled code, deployed as snapshots rather than containers, and governed by exact integer arithmetic rather than approximate floating-point computation.

No prior reading is required. All necessary concepts from the VDR-LLM-Prolog architecture are introduced where first used.

1. The Structural Opportunity

1.1 What Internet Protocols Are

Every internet protocol is a structured conversation between two endpoints. One side sends a message in a defined format. The other side parses the message, processes it, and sends a response in a defined format. HTTP is a request line, headers, and a body, followed by a status line, headers, and a body. SMTP is a sequence of commands — HELO, MAIL FROM, RCPT TO, DATA — each with a defined syntax and expected responses. DNS is a binary packet with a header, question section, and answer section. IRC is one line per message with a prefix, command, and parameters.

These formats are grammars. They have fixed structural elements — delimiters, field positions, encoding rules, framing bytes — and variable content elements — the actual data being communicated. In every protocol, the structural elements are completely deterministic. An HTTP response must have a status line before headers. A JSON body must close every brace it opens. A DNS answer must include the correct count in the header. A SMTP response must start with a three-digit code.

Conventional server software handles both structure and content in compiled code. A developer writes functions to format HTTP responses, serialize JSON, encode DNS packets, frame WebSocket messages. This code is static — it does the same formatting every time, but it must be written, debugged, tested, and maintained.

1.2 What Grammars Provide

In the VDR-LLM-Prolog architecture, a grammar is a persistent template stored in the knowledge base that handles all structural tokens automatically [[HOWL-VDR-12-2026](#)] [[HOWL-VDR-14-2026](#)]. The grammar knows that an HTTP response starts with “HTTP/1.1”, followed by a status code slot, a space, a reason phrase slot, and CRLF. It knows that headers are name-colon-space-value-CRLF. It knows that a blank line separates headers from body. These structural tokens are provided by the grammar at zero cost — no language model forward pass, no token prediction, 100% correct by construction.

The language model fills only the content slots — the status code, the header values, the body content. And even these are often not language model work — they are facts at known addresses in the knowledge base, looked up by Prolog rules and placed into the grammar slots.

This means a protocol handler can be implemented almost entirely without language model token generation. The grammar speaks the protocol. Prolog rules process the data. The language model provides judgment only for requests that require interpretation beyond what stored rules cover.

1.3 What This Paper Demonstrates

Each internet service described in this paper follows the same pattern: a grammar template enforces protocol compliance, Prolog rules over exact arithmetic primitives process

requests, and the knowledge base tree stores all state with full provenance. The services are developed interactively, deployed as snapshots cloned per connection, and improved through rule accumulation. The language model is the runtime that executes stored logic — not a text generator producing server responses through token prediction.

2. Foundational Concepts

2.1 The VDR Triple

Every number in VDR-LLM-Prolog is three integers: Value, Denominator, Remainder [HOWL-VDR-1-2026]. V and D form an exact rational V/D. R is the Remainder — not residual error, but first-class structural information about what the denominator frame could not absorb. When R is zero, the value is a closed rational. When R is nonzero, the value carries exact structure beyond the rational frame. The Remainder is the only slot that nests, the only slot that recurses, the only slot that carries active state.

The system fixes D at 2^{335} , giving 100 decimal digits of precision — 10^{66} times below the Planck length [HOWL-MATH-4-2026]. Addition is one integer addition. Multiplication is one integer multiply plus a bit extraction. The denominator never grows.

This matters for server software because every value the system stores, computes, or compares is exact. A financial transaction amount is a VDR fraction, not a float. A timestamp comparison is exact integer arithmetic. A rate limit threshold is an exact fraction. A metric percentage is an exact ratio of two integers. No rounding surprises. No platform-dependent results.

2.2 The Knowledge Base Tree

A knowledge base (KB) is a structured container with 26 typed fields [HOWL-VDR-5-2026] [HOWL-VDR-8-2026]:

Identity: name, dotted path (root.services.email.users.jane), integer ID.

Persistent state: facts (typed predicates with provenance), rules (Prolog implications), constraints (structural invariants), connections (typed relationships to other KBs), grammars (bidirectional templates), IOSE declarations (interface contracts).

Live state: working data, LRU caches, counters, locks, queues, stacks, ring buffers, bit-sets. All bounded with declared capacity. Cleared by reset, captured by snapshot.

Structural: parent ID, children IDs, mounts.

Metadata: visibility (public/internal/owner-only), frozen flag, owner, timestamps.

KBs form a tree. Every KB has at most one parent and any number of children. The root has no parent. The entire system lives in one tree. Humans address KBs by dotted paths. The runtime uses integer IDs, resolved once and cached. Access to any data is two integers: KB ID plus slot ID. Constant time.

2.3 Scoping and Visibility

When a query executes, the system searches the active KB first, then walks the parent chain to root [[HOWL-VDR-5-2026](#)]. Sibling branches are structurally unreachable — the walk algorithm does not go sideways. This is lexical scoping applied to knowledge. An email user at `root.services.email.users.jane` cannot see `root.services.email.users.bob` — the path never traverses that branch. Not deprioritized. Structurally absent.

Visibility adds a second check. Each KB has a level: public (all users), internal (operators and owners), or owner-only (owning entity). The check is an integer comparison inside the primitive that performs the query. If visibility fails, the KB is skipped entirely — facts absent, not redacted.

2.4 Grants

Every operational primitive — filesystem access, network operations, process management — requires a positive credential grant before execution [[HOWL-VDR-6-2026](#)]. No grant means no operation. A grant specifies: operation class, allowed operations, location constraints, issuer, expiration, maximum uses, remaining uses. Grant state transitions are monotonic — active can become expired, exhausted, or revoked, never re-incremented. Every grant consumption is logged.

2.5 Prolog and Builtins

The Prolog engine provides depth-first search with backtracking over typed terms. Facts are predicates with provenance. Rules are head-body implications. The system's 448 builtin primitives are Prolog predicates — `kb_assert`, `queue_push`, `counter_increment`, `file_read`, `json_parse`, `network_send` are all callable from Prolog rules. A user-written rule chains builtins into a workflow that executes without language model involvement.

2.6 Sessions, Snapshots, and Clones

A snapshot atomically captures all live state — typically 10-500 KB. A clone forks an independent copy from a snapshot. Clones share persistent KBs read-only and have independent live state. Sibling clones cannot see each other's session state — structural impossibility, not access control.

2.7 Grammars

A grammar is a persistent KB field that provides all structural tokens — delimiters, framing, encoding markers — and declares typed content slots. The grammar is bidirectional: it parses structured input into facts and generates structured output from facts. Grammar tokens are free (no language model forward pass) and 100% correct (no malformed output by construction). A grammar for JSON handles all braces, brackets, commas, colons, and quotes. The language model fills only the content values.

2.8 Runner Types

Four runner types serve different execution patterns [HOWL-VDR-20-2026]:

Interactive runners: activated by user input, inherit user’s grants, 50-500 tokens per activation.

Polling runners: timer-driven, spawn fresh each cycle, check conditions and route work, 10-50 tokens per cycle.

Processor runners: maintain persistent data connections, long-lived with periodic respawn at drift thresholds, 8-30 tokens per item.

Internal processing runners: evaluate KB state on schedule, run consistency checks, 20-100 tokens per activation.

2.9 LM Software

LM Software [HOWL-VDR-24-2026] is software developed by configuring language model sessions with structured KB state, encoding logic as Prolog rules over builtins, snapshotting working state as deployable artifacts, and cloning those artifacts on demand. The language model is the runtime. The KB tree is the address space. Prolog is the programming language. Snapshots are the binaries. This paper extends the concept from applications to infrastructure services.

3. The Server Architecture

3.1 Port Listeners

A VDR-LLM-Prolog service is a processor runner with a granted network primitive listening on a port. Each inbound connection spawns a clone from the service’s snapshot. The clone handles the connection — parsing requests through the protocol grammar, processing through Prolog rules, responding through the response grammar — and dies when the connection closes.

The port listener itself is an LM Software application. It was developed interactively — the user taught a session how to speak the protocol, tested it against sample connections, and snapshot it. The snapshot is the server binary. Deployment is running the processor runner on the port.

3.2 Connection Lifecycle

Three connection patterns are configurable per service:

Clone-per-request: each request gets a fresh clone. Maximum isolation. Zero state leakage between requests. Appropriate for stateless APIs. The clone spawns, handles one request, dies.

Clone-per-session: the clone persists for the duration of the TCP connection. Appropriate for protocols with session state — IMAP mailbox selection, FTP working directory, SSH shell session. The clone dies when the connection closes.

Clone-for-N-requests: the clone handles a configurable number of requests, then dies and is replaced. Appropriate for HTTP keep-alive connections where bounded drift is acceptable. N is configured by a rule in the port handler's KB.

The port handler — itself an LM Software application — decides which pattern to use based on rules in its KB. Authenticated users might get persistent sessions. Anonymous users get clone-per-request. High-volume API consumers get clone-for-N with N=50. The decision is a Prolog rule evaluation, not compiled logic.

3.3 Protocol Grammars

Each protocol has a grammar that handles its wire format. The grammar is bidirectional — it parses inbound messages into KB facts and generates outbound messages from KB facts. For text protocols (HTTP, SMTP, IRC, SIP), the grammar handles line structure, header formatting, and body framing. For binary protocols (DNS, MQTT, SNMP, protobuf), the grammar handles byte-level field extraction and construction.

Protocol compliance is structural. The grammar cannot produce a malformed HTTP response because the template enforces the correct field order, required headers, and proper line termination. It cannot produce an invalid DNS packet because the template calculates the correct section counts from the answer facts. The language model never generates protocol framing — the grammar provides it.

3.4 Request Processing

Inbound data arrives at the clone's session KB as facts. An HTTP request becomes facts: `method("GET")`, `path("/api/users/42")`, `header("Authorization", "Bearer xyz")`, `body(Content)`. A DNS query becomes facts: `qname("example.com")`, `qtype(A)`, `qclass(IN)`.

Prolog rules process the facts. A routing rule matches the HTTP path to a handler. An authentication rule validates the bearer token against user KB facts. A DNS lookup rule queries the zone KB for matching records. Each processing step is a Prolog rule invoking builtins — exact, deterministic, auditable.

The response is assembled as facts — status code, headers, body content — and the response grammar fills the protocol template. The complete response is sent through a granted network primitive.

3.5 Scaling

Parallel connections are parallel clones. A hundred simultaneous HTTP requests are a hundred independent sessions. Each has its own KB state. Each processes its own data. They share persistent KBs read-only — configuration, route tables, user databases, zone

files. They share nothing mutable except the data primitives designed for concurrent access — queues with atomic push/pop, counters with atomic increment.

Scaling up is spawning more clones. Scaling down is spawning fewer. A polling runner monitors connection counts (counter) and queue depths and adjusts the clone pool. No load balancer appliance — the port handler distributes connections to clones, and the atomic queue pop distributes work items to workers.

4. Communication Services

4.1 Email: SMTP

A processor runner listens on port 25. Each inbound connection spawns a clone that speaks the SMTP grammar.

The SMTP conversation is a fixed sequence: greeting, EHLO/HELO, MAIL FROM, RCPT TO, DATA, content, QUIT. Each step has defined syntax and expected responses. The grammar provides all protocol framing — the “250 OK” responses, the “354 Start mail input” before DATA, the CRLF.CRLF termination. The clone fills content slots — the greeting hostname, the accepted recipient, the message ID for the Received header.

The clone parses the message during DATA: headers become facts (from, to, subject, date, message_id, each with the raw header value), the body becomes a content fact, attachments become child facts with content-type and encoded content. All facts are asserted to the recipient’s email KB at root.services.email.users.jane.inbox with provenance — received timestamp, source IP, envelope sender.

Before delivery, Prolog rules evaluate in cascade. SPF: a DNS query (through granted network primitive) retrieves the sender’s SPF record, a Prolog rule evaluates the SPF macro language against the connecting IP, the result is a VDR confidence fraction — 1/1 for pass, 0/1 for fail. DKIM: the DKIM-Signature header is parsed, the public key fetched via DNS, the signature verified through a granted cryptographic primitive. DMARC: alignment rules check that SPF and DKIM domains match the From header.

Content analysis uses Prolog rules matching known spam patterns against header and body facts. Bayesian scoring uses VDR fractions — $P(\text{spam}|\text{word})$ as exact fractions derived from training data facts. The spam score is an exact VDR fraction with full provenance showing which rules contributed which scores. A user querying “why was this marked as spam” gets an exact derivation chain, not an opaque classification.

Outbound SMTP: a user’s application asserts message facts to an outbox queue. A processor runner pops from the outbox, resolves the destination MX via DNS query, connects through a granted network primitive, speaks the SMTP grammar outbound, delivers the message, and asserts a delivery receipt fact with provenance.

4.2 Email: IMAP4

A processor runner listens on port 143. Each connection spawns a clone. Authentication checks the user's credentials against their user KB — password hash comparison or certificate validation through a granted cryptographic primitive. On success, the clone loads the user's grants from their scope chain.

The IMAP grammar handles all commands. `SELECT root.services.email.users.jane.inbox` sets the active scope. `FETCH` queries message facts and fills the IMAP response grammar — sequence number, flags, RFC822 content. `STORE` modifies flag facts — read, flagged, deleted. `SEARCH` is a Prolog query against message facts with the IMAP search grammar parsed into query terms. `COPY` creates connection facts linking a message to a destination mailbox KB. `EXPUNGE` retracts messages marked for deletion.

`IDLE`: the clone monitors the user's inbox KB for new facts. When a new message arrives (asserted by the SMTP handler), the clone sends an unsolicited `FETCH` response through the IMAP grammar. The monitoring is a Prolog rule that checks for facts with timestamps newer than the `IDLE` start time.

4.3 IRC

A processor runner connects to an IRC server through a granted network primitive, or acts as an IRC server by listening on port 6667. The IRC protocol is one line per message: optional colon-prefixed source, command, parameters. Perfect for a grammar template.

Each channel is a KB: `root.services.irc.channels.general`. Messages are facts with sender, timestamp, and content. Channel membership is a set of connection facts between user session KBs and the channel KB. `JOIN` asserts a membership connection. `PART` retracts it. `PRIVMSG` to a channel asserts a message fact to the channel KB and pushes the message to each member's outbound queue.

Bot functionality: a `PRIVMSG` directed at the bot triggers a clone that evaluates the message content against Prolog rules. Pattern matching handles known commands — “!help” queries the command registry, “!status” queries monitoring counters, “!deploy” checks grants and triggers a deployment rule. Unknown messages fall through to language model judgment for natural language interpretation. Each successful interpretation can become a new rule, reducing future language model involvement.

Channel moderation is Prolog rules: if message rate from a user exceeds a counter threshold, trigger a `MODE` command via the IRC grammar to mute the user. If a message matches a content filter rule, the message is blocked and a notice is sent. The moderation rules are facts in the channel KB, modifiable by channel operators who have the appropriate grants.

4.4 XMPP/Jabber

XML-based messaging protocol. A processor runner handles the XML stream. The XMPP grammar manages stream negotiation (opening and closing stream tags), SASL authentication stanzas, and the three stanza types: message, presence, and iq (info/query).

Each stanza is parsed by the XML grammar into facts. A message stanza becomes facts: from, to, type, body, thread. Presence stanzas update user state facts — available, away, do-not-disturb. IQ stanzas handle service discovery, roster management, and extension negotiations.

Multi-user chat rooms are channel KBs, structurally identical to IRC channels but with XML framing. XEP extensions — file transfer, message receipts, chat state notifications — are additional grammar templates that activate when the corresponding feature fact is present in the server configuration KB.

4.5 Matrix Protocol

HTTP-based federation. Each Matrix room is a KB. Events are facts — membership, messages, state changes — with provenance including the event’s cryptographic hash. The directed acyclic graph structure of Matrix events maps to the KB tree: each event fact references parent event facts through connection relationships.

Federation: a processor runner handles inbound federation requests on port 8448. Each request is an HTTP POST with a signed JSON body. Signature verification uses exact VDR arithmetic through granted cryptographic primitives. The signing key is a fact in the server’s security KB with owner-only visibility.

End-to-end encryption: Olm/Megolm session keys are facts in user KBs with owner-only visibility. Key exchange events pass through the system as opaque encrypted payloads. Only the recipient’s clone, with the appropriate grants and keys, can decrypt.

4.6 Slack and Discord

Both use HTTP/WebSocket hybrid protocols. Slack events arrive as HTTP POSTs with JSON bodies. Discord events arrive as WebSocket frames with JSON payloads. In both cases, the JSON grammar parses inbound events into facts.

Workspaces and guilds are KBs. Channels are child KBs. Messages are facts. Threads are child KBs of the message that started them. Reactions are facts with emoji, user, and timestamp.

Slack blocks and Discord embeds — structured message formats — are grammar templates. Section blocks, action blocks, context blocks are filled from KB data. Interactive components — buttons, menus, date pickers — trigger HTTP callbacks that spawn clones to handle the interaction. Each interaction is a Prolog rule evaluation: button “approve” triggers `approve_request(RequestID)`, which updates the request KB and sends a confirmation block back through the response grammar.

4.7 ActivityPub/Mastodon

Federation protocol using HTTP signatures and JSON-LD. Each actor is a KB with inbox and outbox queues. Inbound activities arrive as HTTP POSTs — the processor runner verifies the HTTP signature through granted cryptographic primitives, parses the activity JSON, and asserts the object as a fact in the appropriate KB.

Outbound activities: posting content asserts a Create activity fact to the outbox queue. A processor runner pops from the queue, formats the activity using the ActivityPub grammar, signs it with the server's key, and POSTs it to each follower's inbox through granted network primitives. The follower list is a set of connection facts.

5. Web Services

5.1 HTTP/HTTPS Server

A processor runner listens on port 80 or 443. Each request spawns a clone. The HTTP request grammar parses method, path, headers, and body into facts. TLS handshake is handled by a granted network primitive using certificate and key facts from the server's security KB.

Routing is a Prolog rule: `route("/api/users", handle_users). route("/api/orders", handle_orders). route("/static/*", serve_static).` Each handler is a Prolog rule chain that queries KBs, processes data with builtins, and fills an HTTP response grammar. HTML responses use page grammars with content slots. JSON API responses use the JSON grammar. Static files resolve to file facts and are served through granted file read primitives.

Session management: cookies map to session KB paths. Set-Cookie carries the session identifier. Subsequent requests mount the session KB. Session expiration is a counter — idle time incremented per cycle by a polling runner, session KB retracted when the counter exceeds the timeout threshold.

HTTP/2: multiplexed streams over a single connection. Each stream is a lightweight child context within the clone. The HTTP/2 frame grammar handles HEADERS, DATA, SETTINGS, WINDOW_UPDATE, PING frames. Stream priorities are VDR fractions governing the share of bandwidth per stream.

5.2 WebSocket

The HTTP handler upgrades the connection on a 101 Switching Protocols response. The WebSocket frame grammar handles opcode (text, binary, close, ping, pong), payload length, masking, and payload. Inbound frames become facts. Outbound frames are grammar-filled and sent through the socket.

Ping/pong is automatic: a Prolog rule responds to ping frames with pong frames carrying the same payload. Close handshake follows the RFC — echo the close frame, then terminate.

Real-time applications — chat, notifications, live updates — push outbound frames from queue contents. A mutation elsewhere in the KB tree pushes a notification fact to the user's update queue. The WebSocket clone pops from the queue and sends the frame.

5.3 GraphQL

An HTTP endpoint where the request body contains a GraphQL query. The GraphQL grammar parses queries, mutations, and subscriptions into an abstract syntax tree represented as KB facts. The schema is a KB of type definitions — each type is a KB, each field is a fact with type, arguments, and resolver rule reference.

Field resolution is a Prolog rule: `resolve(User, email, _, Email) :- user_email(User, Email)`. Nested resolution is natural — Prolog resolves fields recursively, each level querying the appropriate KB. Batching addresses N+1 problems: a Prolog rule collects all IDs at one level and issues a single batch query before resolving children.

Subscriptions use queues. A mutation that changes data pushes a notification to subscriber queues. The WebSocket handler pops from the queue and sends the update through the GraphQL response grammar.

5.4 gRPC

HTTP/2 based RPC with protobuf serialization. The protobuf schema compiles to a grammar — each message type is a grammar template with typed fields. Service definitions become Prolog rules mapping method names to handlers.

Streaming RPCs use queues. Server streaming: the handler pushes response items to an outbound queue that the HTTP/2 framing drains. Client streaming: inbound frames populate a request queue that the handler pops from. Bidirectional: both queues simultaneously.

Protobuf field types map to VDR types: `int32` and `int64` are integer facts, `float` and `double` become VDR fractions at the boundary with the conversion precision logged, `string` is an interned string ID, `bytes` is a raw buffer, repeated fields are list facts, nested messages are child KB facts.

5.5 REST API with JSON

The most common case. An HTTP handler receives JSON payloads. The JSON grammar parses the payload into typed facts — strings, numbers (as VDR fractions), booleans, nulls, arrays, nested objects. A Prolog rule chain validates the input against a schema KB, processes the request, queries or mutates the data KBs, and fills a JSON response grammar.

Pagination: a Prolog rule returns facts within an offset/limit range. The next page link includes the offset as a query parameter. Filtering: query parameters become Prolog query terms. Sorting: collection sort builtins with VDR fraction comparison for numeric fields.

Error responses use an error grammar template — HTTP status code, JSON body with error code, message, and details. The grammar ensures consistent error formatting across all endpoints.

5.6 RSS/Atom Feed Generation

An HTTP endpoint serving XML. The Atom grammar fills the feed envelope from blog or news KB facts. Each entry is a recent content fact formatted into the entry grammar. A Prolog rule selects the most recent N entries by timestamp comparison. ETag support: the ETag header is a checksum of the feed's current fact set. Conditional GET returns 304 Not Modified when unchanged. Zero language model tokens — purely grammar-driven from KB facts.

6. Infrastructure Services

6.1 DNS

A processor runner listens on port 53, UDP and TCP. The DNS wire format is a binary grammar — the 12-byte header with ID, flags, question count, answer count, authority count, and additional count, followed by the question and response sections.

Zone data maps to the KB tree. `root.services.dns.zones.example_com` contains record facts: `a_record("www", "192.168.1.1", 3600)`. `mx_record("@", 10, "mail.example.com", 3600)`. `cname_record("blog", "www.example.com", 3600)`. The TTL is an integer on each fact.

A query clone parses the question section — QNAME, QTYPE, QCLASS — and queries the zone KB with a Prolog rule. Matching facts fill the answer section of the response grammar. The answer count in the header is computed from the number of matching facts.

Recursive resolution: if the zone KB lacks the answer, a forwarding rule queries an upstream resolver through a granted network primitive, caches the response in an LRU with TTL-based expiration (counter comparison: current time minus cache timestamp versus TTL), and returns it.

Zone transfers (AXFR): a Prolog query collects all facts in the zone KB, fills a sequence of DNS response records, and sends them in a TCP stream. The zone serial number is a counter — incremented on any zone modification.

6.2 DHCP

A processor runner listens on port 67 UDP. The DHCP packet grammar handles the fixed-format fields — op, htype, hlen, hops, xid, flags, addresses — and the variable options field.

The IP address pool is a KB of address facts. Each address has a state: available, offered, or leased. A DISCOVER triggers a Prolog rule that finds an available address, asserts an offered state with the client's MAC and a lock, and sends an OFFER via the response grammar. A REQUEST confirms the lease — the lock is released, the state becomes leased with an expiration counter. A polling runner checks lease counters and retracts expired leases, returning addresses to available state.

Options — subnet mask, gateway, DNS servers, domain name — are facts in the DHCP configuration KB. The options field in the response grammar is filled from these facts.

6.3 NTP

A processor runner listens on port 123 UDP. The NTP packet is a 48-byte binary grammar — leap indicator (2 bits), version (3 bits), mode (3 bits), stratum, poll interval, precision, root delay, root dispersion, reference ID, and four 64-bit timestamps.

The server's reference time is a fact updated by a hardware clock primitive or upstream NTP query. Client requests receive a response with transmit timestamp, receive timestamp, and reference timestamp filled from clock facts. All timestamps are exact 64-bit fixed-point integers — NTP's native format. VDR represents this losslessly. No float conversion at any point.

6.4 SSH

A processor runner listens on port 22. The SSH transport grammar handles key exchange (Diffie-Hellman via granted cryptographic primitives), user authentication (public key lookup in user KB, password hash comparison), and channel multiplexing.

Shell sessions: a clone per authenticated connection provides a command interface. Commands are parsed by a shell grammar and matched against Prolog rules. `ls` maps to `list_dir`. `cat` maps to `file_read`. Custom commands map to application-specific rules at the user's scope. Output is formatted by a terminal grammar.

SCP/SFTP subsystems: file operations through the same granted filesystem primitives as FTP, negotiated through SSH channel requests.

Port forwarding: TCP connections tunneled through SSH channels. Authorization for forwarding is a grant fact on the user's session. The clone acts as a proxy, wrapping and unwrapping traffic in SSH channel frames.

6.5 LDAP

A processor runner listens on port 389. LDAP uses BER/DER encoded ASN.1 — a binary grammar that handles tag-length-value encoding. BIND authenticates against user KBs. SEARCH queries the directory tree.

The LDAP directory maps directly to the KB tree. A distinguished name “`cn=Jane Doe,ou=Engineering,dc=acme,dc=com`” maps to `root.org.acme.engineering.users.jane_doe`. SEARCH with filter “`(objectClass=person)`” is a Prolog query across the subtree for facts matching the `objectClass` predicate.

ADD, MODIFY, DELETE map to `kb_assert`, fact modification, and `kb_retract`. Schema enforcement is constraints on the directory KB — required attributes, allowed object classes, syntax validation. All existing LDAP-aware applications — email clients, VPN authenticators, SSH key lookups — work against this service without modification.

6.6 RADIUS/TACACS+

Network device authentication, authorization, and accounting. A processor runner listens on port 1812 (authentication) and 1813 (accounting). The RADIUS packet grammar handles Access-Request, Access-Accept, Access-Reject.

Authentication checks credential facts in user KBs. Authorization checks grants — which network segments the user can access, which privilege levels. Accounting-Request messages become facts in the user's session KB — timestamps, bytes transferred, session duration as VDR fractions with exact arithmetic. No rounding ambiguity in accounting records.

7. File and Object Storage

7.1 FTP/SFTP

A processor runner listens on port 21 (FTP) or 22 (SFTP). Clone per connection, authenticated against user KBs. The FTP command grammar handles USER, PASS, LIST, RETR, STOR, CWD, MKD, DELE, RMD.

Each command maps to a Prolog rule invoking granted filesystem primitives. LIST calls `list_dir` and fills the directory listing grammar. RETR calls `file_read` and streams content. STOR calls `file_write`. CWD changes the active path as a working data binding in the session.

The grant system provides access control. A user's grants specify which filesystem paths they can read and write. RETR on a path outside their granted scope is denied by the primitive layer before any data is accessed.

7.2 S3-Compatible Object Storage

An HTTP endpoint implementing the S3 REST API. PUT creates a KB for the object — content stored via a granted filesystem primitive, metadata as facts (content-type, content-length, ETag as checksum). GET retrieves content and fills HTTP headers from metadata facts. LIST on a bucket is a Prolog query across child KBs matching the prefix parameter.

Multipart upload: a temporary KB per upload with part facts. `CompleteMultipartUpload` merges parts and retracts the temporary KB. Versioning: each PUT creates a new version KB as a sibling, version ID is the KB's integer ID.

ACLs are grant facts. Pre-signed URLs are temporary grant facts with expiration — a token in the URL maps to a fact authorizing one operation within the time window.

7.3 WebDAV

HTTP extension for collaborative authoring. The WebDAV grammar handles PROPFIND, PROPPATCH, MKCOL, COPY, MOVE, LOCK, UNLOCK. Each resource is a KB. Properties are facts. Collections are parent KBs with children. LOCK

uses the lock primitive — acquire, record the lock token as a fact, release on UNLOCK or timeout. COPY is clone. MOVE is clone plus retract original.

8. Authentication and Authorization

8.1 OAuth 2.0 / OpenID Connect

The authorization server is an LM Software application. The authorization endpoint presents a consent page using an HTML grammar template, validates the `client_id` against registered client facts, and generates an authorization code — a random integer stored as a temporary fact with an expiration counter.

The token endpoint exchanges the code for an access token — another integer fact on the user's session KB with scope facts listing permitted operations. Token validation: any service receiving a token queries the authorization server's token KB and gets back scope facts. Refresh tokens are longer-lived facts with rotation — each use retracts the old token and asserts a new one.

Grant state transitions are monotonic — issued, used, revoked, never un-revoked. The same monotonic grant model that governs VDR-LLM-Prolog system grants [[HOWL-VDR-6-2026](#)] governs OAuth tokens.

8.2 SAML

The identity provider is an LM Software application. User authenticates via credential check. The IdP clone generates a SAML assertion using an XML grammar template — Issuer, Subject, Conditions, AttributeStatement filled from user KB facts. The assertion is signed through a granted cryptographic primitive. Service providers validate the signature and extract attributes. Each SAML attribute maps to a user KB fact — email, groups, roles.

8.3 LDAP-Backed SSO

Combines the LDAP directory service (§6.5) with SAML or OIDC. User authenticates via LDAP BIND. The identity provider queries the LDAP KB for user attributes, generates the appropriate token or assertion. Group membership in the LDAP tree drives grant inheritance — user in group “engineering” inherits engineering branch grants automatically through the KB tree's scope mechanism.

9. Data Services

9.1 Redis-Compatible Cache

A processor runner listens on port 6379. The RESP (Redis Serialization Protocol) grammar handles the wire format — simple strings, errors, integers, bulk strings, arrays.

GET is a fact lookup. SET is a fact assertion with optional TTL as an expiration counter. DEL retracts the fact. EXPIRE sets a TTL counter. A polling runner checks counters and retracts expired facts.

Redis data types map to KB primitives: lists are queues, sets are set primitives, sorted sets use facts with score as VDR fractions — ZRANGEBYSCORE is a Prolog query with exact fraction comparison. Pub/sub uses queue-based channels — SUBSCRIBE registers a connection, PUBLISH pushes to all subscriber queues.

Every operation has provenance. Every key has an audit trail. Sorted set scores are exact fractions, not floating-point doubles.

9.2 Kafka-Compatible Streaming

A processor runner handles the Kafka wire protocol. Topics are KBs. Partitions are child KBs. Each message is a fact with an offset (monotonically incrementing counter), key, value, and timestamp.

Producers append facts — the offset counter increments, never reuses. Consumers track position via consumer group KBs with committed offset facts per partition. Fetch requests collect facts with offset $\geq$ committed offset up to the fetch limit.

Retention: a polling runner retracts message facts older than the retention policy (a constraint on the topic KB). Compaction: for each key, retain only the latest fact, retract older ones.

9.3 SQL-Compatible Query Interface

An HTTP or TCP endpoint accepting SQL queries. The SQL grammar parses SELECT, INSERT, UPDATE, DELETE into Prolog query terms. Tables map to KBs. Rows are facts. Columns are fact arguments. SELECT becomes a Prolog query with the WHERE clause translated to Prolog goals. JOIN is a multi-KB query with shared variables. ORDER BY uses collection sort builtins. GROUP BY uses aggregation rules with VDR fraction arithmetic for SUM, AVG, COUNT.

INSERT is kb_assert. UPDATE is retract plus assert. DELETE is kb_retract. Transactions use locks — acquire locks on affected KBs, perform operations, release. ACID properties derive from the KB's structural properties: atomicity through lock-protected batches, consistency through constraints, isolation through clone independence, durability through persistent fact storage.

10. Monitoring and Observability

10.1 Prometheus/OpenMetrics

A processor runner serves the /metrics endpoint. Each metric is a fact — name, type (counter/gauge/histogram/summary), labels as path components, value as VDR fraction.

A scrape request triggers a Prolog query that collects all metric facts and fills the OpenMetrics text grammar.

Alerting rules are Prolog with exact arithmetic: `alert(high_cpu) :- metric(cpu_usage, Host, Value), Value > 80/100`. The threshold is exact. The comparison is exact. The alert fires or it does not.

Recording rules compute derived metrics: `request_rate(Service, Rate) :- counter_value(requests_total, Service, Current), counter_value_previous(requests_total, Service, Previous), Rate is (Current - Previous) / interval`. Rate as a VDR fraction — exact.

10.2 Syslog

A processor runner listens on port 514 UDP or 6514 TCP with TLS. The syslog grammar (RFC 5424) parses priority, version, timestamp, hostname, app-name, procid, msgid, structured data, and message into facts.

Log analysis rules fire on ingestion: if facility is local0 and severity is 3 or below and the message contains “disk,” push to the `storage_alerts` queue. Aggregation uses counters — errors per hour per host. Correlation is Prolog queries across host log KBs — which hosts reported disk errors within 5 minutes of each other, resolved by timestamp comparison with VDR arithmetic.

10.3 SNMP

A processor runner listens on port 161 UDP. The SNMP grammar handles BER-encoded ASN.1 packets. The MIB maps to a KB tree — each OID is a path, each value is a fact. OID 1.3.6.1.2.1.1.1 (`sysDescr`) is a fact at `root.services.snmp.mib.system.sysDescr`.

GET retrieves facts by OID path. GETNEXT walks the tree. GETBULK retrieves ranges. SET asserts facts with grant checks. TRAP/INFORM notifications push to a queue that a trap receiver processes.

10.4 Distributed Tracing

Each service clone generates trace facts — trace ID, span ID, parent span ID, service name, operation name, start time, duration as VDR fraction, status, tags. Trace facts are asserted to a tracing KB. A query for “show me the trace for request X” is a Prolog query that collects all spans with the same trace ID, sorts by start time, and computes the critical path. Latency percentiles are exact VDR fractions computed from span duration facts.

11. IoT and Messaging

11.1 MQTT

Lightweight pub/sub for IoT. A processor runner listens on port 1883. The MQTT packet grammar handles CONNECT, PUBLISH, SUBSCRIBE, UNSUBSCRIBE, PIN-

GREQ/PINGRESP.

Topics are KB paths. `root.services.mqtt.sensors.building_a.floor_3.temperature` holds readings as facts. PUBLISH writes a fact with payload, QoS level, and timestamp. SUBSCRIBE registers a connection between the subscriber's session KB and the topic KB. When a PUBLISH arrives, a Prolog rule finds subscribers through connection queries and pushes the message to each subscriber's outbound queue.

QoS levels map to coordination primitives. QoS 0 (at most once): fire and forget. QoS 1 (at least once): PUBACK handshake with a delivery counter. QoS 2 (exactly once): four-step handshake with locks ensuring the message is delivered once and only once.

Retained messages are persistent facts on the topic KB. Last Will messages are facts on the client's session KB that a Prolog rule asserts to the specified topic on disconnect.

11.2 AMQP

Advanced Message Queuing Protocol. A processor runner handles the AMQP frame grammar — method, header, body, and heartbeat frames. Exchanges are KBs with routing rules. Queues are queue primitives on KBs. Bindings are connection facts between exchange KBs and queue KBs with routing key patterns.

Publishing sends a message to an exchange KB. The exchange's routing rules (Prolog) determine which bound queues receive the message. Direct exchange: routing key equals binding key. Topic exchange: routing key matches binding pattern with wildcards. Fanout: all bound queues. Headers exchange: message header facts matched against binding header patterns.

Consumer acknowledgment uses counters — delivery tag increments per message, ack/nack retracts or requeues.

11.3 CoAP

Constrained Application Protocol for IoT devices. UDP-based, binary format. The CoAP message grammar handles the 4-byte header (version, type, token length, code, message ID) and options (delta-encoded). Request/response semantics mirror HTTP but with much smaller packets.

Resource discovery: the `/.well-known/core` endpoint returns a CoRE Link Format grammar listing all resources as KB paths with attributes. Observe: a subscription mechanism similar to MQTT — the server pushes notifications when resource facts change, tracked by a sequence counter.

12. Media and Real-Time

12.1 Video Streaming: HLS/DASH

An HTTP endpoint serving manifest files and media segments. The HLS M3U8 grammar fills the manifest template — EXT-X-VERSION, EXT-X-TARGETDURATION, EXTINF per segment with duration as VDR fraction. Each segment is a fact: URL, duration, sequence number.

Adaptive bitrate: multiple rendition KBs (720p, 1080p, 4K). The master manifest lists all renditions. The client selects based on bandwidth. Segment files are served through granted filesystem primitives.

Live streaming: a processor runner ingests the live feed through a granted media primitive, segments it, asserts new segment facts, retracts segments beyond the sliding window. The manifest regenerates from current segment facts on each request. DVR mode: retain all segments, extend the window.

12.2 SIP/RTP (VoIP)

SIP: a processor runner listens on port 5060. The SIP grammar handles INVITE, ACK, BYE, REGISTER, OPTIONS. Each call is a session KB with call state tracked as a Prolog state machine — idle, ringing, connected, terminated. SDP content within SIP is a nested grammar with media type, codec, and port facts.

RTP: media streams flow as raw UDP. A processor runner handles the 12-byte RTP header grammar — version, padding, extension, CSRC count, marker, payload type, sequence number, timestamp, SSRC. Audio/video payload passes through without language model involvement. Jitter buffer is a ring buffer. Packet loss detection is a bitset tracking received sequence numbers. RTCP statistics — jitter, round-trip time, loss fraction — are VDR fractions.

12.3 WebRTC Signaling

WebSocket-based signaling for peer-to-peer media. The signaling grammar handles offer/answer SDP exchange and ICE candidate trickling as JSON messages. Each peer connection is a session KB with local SDP, remote SDP, ICE candidates, and connection state facts. The media flows peer-to-peer — VDR-LLM-Prolog handles only the signaling, not the media transport.

TURN relay fallback: a processor runner handles TURN allocation requests. Each allocation is a session KB with permissions, channels, and lifetime as a counter with expiration.

13. Print and Legacy Services

13.1 IPP (Internet Printing Protocol)

A processor runner listens on port 631. IPP is HTTP POST with binary-encoded attributes. The IPP grammar parses job attributes — document format, copies, page ranges, finishing options. Each job is a KB with state facts: pending, processing, completed, aborted. The printer is a KB with capability facts. A Prolog rule validates the job against capabilities before accepting. Job processing invokes granted printer primitives.

13.2 Telnet

A processor runner listens on port 23. Raw TCP with optional TELNET negotiation. A clone per connection presents a text interface. Input lines are parsed by a command grammar. Output uses text templates. This pattern supports MUDs, BBS systems, text-based management interfaces, and equipment consoles — all as LM Software applications.

13.3 Legacy Terminal Protocols (3270/5250)

Mainframe and midrange terminal protocols. A processor runner speaks the 3270 or 5250 data stream through a granted network primitive. The terminal screen grammar maps field positions to KB facts. Each screen is a template with input and output fields at fixed positions. Navigation — function keys, field tabbing, screen transitions — is encoded as Prolog rules. This enables the COBOL mainframe and AS/400 RPG integration scenarios described in [\[HOWL-VDR-24-2026\]](#), where a modern JSON API is provided by a translation clone that drives legacy terminal sessions through stored Prolog rules.

14. Calendar, Contacts, and Collaboration

14.1 CalDAV/CardDAV

WebDAV-based protocols for calendars and contacts. The iCalendar grammar parses and generates VEVENT, VTODO, VJOURNAL components. Each event becomes a KB fact with start time, end time, summary, location, recurrence rules, attendees.

Recurrence rules are Prolog: a recurring daily event generates occurrence facts for each date. A query for “events this week” evaluates recurrence rules and returns both one-time and recurring events. Conflict detection checks for overlapping time ranges using VDR fraction comparison on timestamps.

Free/busy queries: a Prolog query across the user’s calendar KB collecting all event time ranges and returning the complement. Shared calendars are mounted KBs — mount a colleague’s calendar read-only to see availability without seeing event details.

Contacts (CardDAV): each contact is a KB fact with name, email, phone, address. The vCard grammar handles import/export. Search is Prolog — find contacts matching partial name, company, or tag.

14.2 WebDAV with Collaborative Editing

Multiple users editing shared documents. Each document is a KB. Edits are facts with author, timestamp, and change description. LOCK prevents concurrent modification. Versioning creates child KBs for each saved version. Diff between versions is a structural operation on two sets of facts.

15. Security Services

15.1 Certificate Authority

An LM Software application handling certificate signing requests. The CSR grammar parses the request. Validation rules check the requester's identity against organizational KBs. The signing operation uses a granted cryptographic primitive with the CA's private key stored in an owner-only KB. Certificate facts — subject, issuer, serial number, validity period, public key — are asserted to a certificate KB with provenance. Certificate Revocation Lists are generated by querying for revoked certificate facts and filling the CRL grammar.

15.2 OCSP Responder

An HTTP endpoint answering certificate validity queries. The OCSP grammar parses the request (certificate serial number), queries the certificate KB for revocation facts, and fills the OCSP response grammar with the status — good, revoked, or unknown. Signed with the responder's key through a granted cryptographic primitive.

15.3 Firewall / Packet Filter API

A JSON API managing firewall rules. Each rule is a KB fact — source, destination, port, protocol, action (allow/deny), priority as integer. Rule evaluation is a Prolog query ordered by priority — first matching rule wins. Rule modification through the API requires grants. Changes are logged with provenance. A polling runner applies the rule set to the packet filter through granted system primitives.

16. The Structural Advantages

16.1 Protocol Compliance by Construction

Grammars cannot produce malformed protocol output. An HTTP response always has a valid status line. A DNS packet always has correct section counts. A JSON body always has balanced braces. A SMTP conversation always follows the correct command sequence. This eliminates an entire class of server bugs — malformed responses, missing headers, encoding errors, framing violations — that consume significant development and debugging effort in conventional server software.

16.2 Security Without Middleware

Conventional servers implement security as middleware — authentication modules, authorization checks, rate limiters, WAF rules. Each is a software layer that must be written, configured, and maintained. Each can be misconfigured. Each adds latency.

In VDR-LLM-Prolog, security is the KB tree. Authentication verifies credentials against user KB facts. Authorization checks grants on the primitives. Rate limiting is a counter comparison. Access control is scope visibility — an integer comparison. Audit logging is automatic provenance. No middleware. No configuration files. No security as an afterthought.

16.3 Exact Accounting

Financial transactions, billing records, rate calculations, SLA measurements — anything involving numbers — use exact VDR fractions. A transaction of \$10,000.00 is 1000000/100, not a float. A 99.99% SLA is 9999/10000, not 0.9999. A 2.5% tax rate is 25/1000, not 0.025. No rounding errors in billing. No float precision disputes. No platform-dependent results.

16.4 Complete Audit Trail

Every operation on every service is logged with full provenance — who, what, when, through which rule, against which data, with what result. A regulator asking “what happened to this transaction six months ago” gets an exact answer from a query against the audit KB. Not a log search. Not a reconstruction. An exact provenance chain at integer addresses.

16.5 Improvement Through Usage

Every service accumulates rules. An HTTP API that handles a new request pattern through language model judgment can encode the pattern as a Prolog rule. Future requests of that type are handled at level 3 — zero language model tokens. A spam filter that encounters a new spam pattern can add a detection rule that fires automatically on future messages. An IRC bot that learns a new command adds a rule. Each service gets cheaper and more capable through usage.

16.6 Update Without Redeployment

Changing a DNS record is asserting one fact. Changing a firewall rule is retracting one fact and asserting another. Changing an authentication policy is modifying a Prolog rule. Updating an SSL certificate is replacing a fact in the security KB. Every change takes immediate effect for all future clones. No server restart. No container redeployment. No rolling update. One fact at one integer address.

17. Development Workflow

17.1 Building a Service

The developer starts an interactive session. They describe the protocol the service should speak. They provide sample requests and expected responses. The session learns to handle each request type — parsing through the grammar, processing through Prolog rules, responding through the response grammar.

For standard protocols, the grammar handling is often immediate — HTTP, JSON, XML, and line-based protocols have well-known structures. The development effort is in the processing logic: routing rules, data access patterns, business logic, error handling.

The developer tests against sample traffic. When the session handles all expected request types correctly, they snapshot it. The snapshot is the server.

17.2 Deploying a Service

The developer configures a processor runner to listen on the appropriate port with the appropriate grants — network listen, filesystem access, database access, whatever the service requires. The port handler uses the snapshot as its clone source. The service is live.

17.3 Monitoring a Service

A polling runner checks service health: connection counter (active sessions), request counter (throughput), error counter (error rate), queue depths (backlog). Alerting rules are Prolog with exact threshold comparisons. A dashboard is an HTTP service — another LM Software application — that queries the monitoring KBs and fills an HTML grammar template.

17.4 Updating a Service

Data changes: modify facts in persistent KBs. Immediate effect. Logic changes: retract old rules, assert new ones. Immediate effect. Protocol changes: develop a new session with the updated handling, snapshot it, update the port handler's clone source. New connections get the new version. Existing connections continue with the old version until they close.

Canary deployment: the port handler routes a percentage of connections to clones from the new snapshot, the remainder to clones from the old snapshot. A monitoring rule compares error rates between versions using exact VDR fraction comparison. Auto-rollback if the new version exceeds the threshold.

18. Limitations

18.1 Raw Performance

Each clone is a language model session. Even when operating at level 3 (pure Prolog, zero language model tokens), the Prolog evaluation runs within the VDR-LLM-Prolog runtime, which is slower per operation than compiled C or Rust server code. For high-throughput, low-latency services — millions of requests per second with sub-millisecond response times — conventional compiled servers remain superior.

VDR-LLM-Prolog services are appropriate where request rates are moderate (thousands to tens of thousands per second), where correctness and auditability matter more than raw throughput, and where the development speed advantage (hours versus weeks) justifies the per-request overhead.

18.2 Binary Protocol Complexity

Some binary protocols have complex state machines, variable-length encodings, and context-dependent parsing that stretch the grammar system. TLS handshake negotiation, HTTP/2 HPACK header compression, and protobuf varint encoding are handled through specialized grammar extensions and granted cryptographic primitives rather than pure grammar templates.

18.3 Media Processing

Audio/video encoding, decoding, and transcoding are compute-intensive operations not suited for language model execution. VDR-LLM-Prolog services handle the signaling and control plane for media services (SIP, WebRTC signaling, HLS manifests) but delegate actual media processing to granted external primitives or conventional media engines.

18.4 Stateful Protocol Complexity

Protocols with complex stateful semantics — BGP route convergence, OSPF link-state databases, distributed consensus (Raft/Paxos) — require careful state machine design in Prolog. The state machine is expressible but the development investment is higher than for request/response protocols. Each state transition must be encoded as a rule, tested, and validated.

18.5 Language Model Dependency

Every clone requires a language model runtime, even when operating at level 3 with zero language model token generation. The runtime provides the Prolog evaluation engine, the grammar system, and the KB access primitives. This is a heavier base than a compiled binary. Memory footprint per clone is larger than per thread in a conventional server. The tradeoff is development speed and auditability versus resource efficiency.

Appendices

Appendix A — Protocol Grammar Classification

Protocol	Wire format	Grammar type	Structural token %	Content handled by
HTTP/1.1	Text lines	Line grammar	~40%	Prolog rules + KB facts
HTTP/2	Binary frames	Binary grammar	~60%	Frame decoder + Prolog
SMTP	Text lines	Line grammar	~55%	Prolog rules
IMAP4	Text lines	Line grammar	~50%	Prolog rules + KB queries
POP3	Text lines	Line grammar	~60%	Prolog rules + KB queries
IRC	Text lines	Line grammar	~45%	Prolog rules
XMPP	XML stream	XML grammar	~65%	Prolog rules
DNS	Binary packet	Binary grammar	~70%	Zone KB lookup
DHCP	Binary packet	Binary grammar	~65%	Pool KB + Prolog rules
NTP	Binary packet	Binary grammar	~80%	Clock facts
MQTT	Binary packet	Binary grammar	~60%	Topic KB + subscriber queries
SSH	Binary frames	Binary grammar	~55%	Auth KB + granted primitives
LDAP	BER/DER ASN.1	Binary grammar	~65%	Directory KB tree
SIP	Text headers	Line grammar	~45%	Call state Prolog
RTP	Binary header	Binary grammar	~85%	Payload passthrough
FTP	Text commands	Line grammar	~50%	Filesystem primitives
SNMP	BER/DER ASN.1	Binary grammar	~70%	MIB KB tree
RADIUS	Binary packet	Binary grammar	~60%	User KB + grants
gRPC	HTTP/2 + protobuf	Nested binary	~55%	Schema KB + handler rules
GraphQL	HTTP + custom	Query grammar	~30%	Schema KB + resolver rules

Protocol	Wire format	Grammar type	Structural token %	Content handled by
WebSocket	Binary frames	Frame grammar	~75%	Application-dependent
RESP (Redis)	Text protocol	Line grammar	~40%	Cache KB operations
Kafka	Binary protocol	Binary grammar	~55%	Topic KB + offset tracking
CoAP	Binary compact	Binary grammar	~75%	Resource KB
AMQP	Binary frames	Frame grammar	~60%	Exchange routing rules
Syslog	Text (RFC 5424)	Line grammar	~35%	Log KB + analysis rules
HLS/M3U8	Text manifest	Line grammar	~50%	Segment facts
CalDAV	HTTP + iCalendar	Nested text	~55%	Calendar KB
IPP	HTTP + binary attrs	Binary grammar	~65%	Job KB + printer capabilities
SAML	HTTP + XML	XML grammar	~70%	User attributes + signing
OAuth	HTTP + JSON	JSON grammar	~45%	Token KB + scope facts
ActivityPub	HTTP + JSON-LD	JSON grammar	~40%	Actor KB + federation rules
OpenMetrics	Text exposition	Line grammar	~55%	Metric facts

Appendix B — Port Assignment Table

Port	Protocol	Runner type	Clone pattern	Typical tokens per request	Grant requirements
22	SSH	Processor	Clone-per-session	30-100 (shell), 8-15 (SCP)	Network listen, filesystem, crypto
23	Telnet	Processor	Clone-per-session	20-50	Network listen

Port	Protocol	Runner type	Clone pattern	Typical tokens per request	Grant requirements
25	SMTP	Processor	Clone-per-connection	15-30	Network listen, network send (outbound), crypto (DKIM)
53	DNS	Processor	Clone-per-query (UDP)	8-12	Network listen, network send (recursion)
67	DHCP	Processor	Clone-per-request (UDP)	8-15	Network listen
80	HTTP	Processor	Clone-per-request or clone-for-N	15-50 (API), 8-12 (static)	Network listen, filesystem (static), varies by handler
110	POP3	Processor	Clone-per-session	10-20	Network listen
123	NTP	Processor	Clone-per-request (UDP)	8	Network listen
143	IMAP	Processor	Clone-per-session	10-25 per command	Network listen
161	SNMP	Processor	Clone-per-request (UDP)	8-12	Network listen
389	LDAP	Processor	Clone-per-session	10-20 per operation	Network listen
443	HTTPS	Processor	Clone-per-request or clone-for-N	15-50 (API), 8-12 (static)	Network listen, crypto (TLS)
514	Syslog	Processor	Clone-per-message (UDP) or persistent (TCP)	8-15	Network listen

Port	Protocol	Runner type	Clone pattern	Typical tokens per request	Grant requirements
631	IPP	Processor	Clone-per-job	15-25	Network listen, printer access
993	IMAPS	Processor	Clone-per-session	10-25 per command	Network listen, crypto
1812	RADIUS auth	Processor	Clone-per-request	10-15	Network listen
1813	RADIUS acct	Processor	Clone-per-request	8-12	Network listen
1883	MQTT	Processor	Clone-per-session	8-15 per publish	Network listen
5060	SIP	Processor	Clone-per-call	20-40	Network listen, network send
5222	XMPP	Processor	Clone-per-session	10-20 per stanza	Network listen, crypto
5672	AMQP	Processor	Clone-per-session	10-20 per message	Network listen
6379	Redis compat	Processor	Clone-per-session	8-10 per command	Network listen
6667	IRC	Processor	Clone-per-session	10-20 per message	Network listen
8443	Federation	Processor	Clone-per-request	15-30	Network listen, crypto
8448	Matrix fed	Processor	Clone-per-request	15-30	Network listen, crypto
9090	Metrics	Processor	Clone-per-scrape	15-30	Network listen
9092	Kafka compat	Processor	Clone-per-session	10-15 per message	Network listen

Appendix C — KB Tree Layout for a Complete Service Stack

root

- └─ system
 - | └─ oso (operational principles as ~176 Prolog terms)
 - | └─ audit (append-only, axiom-protected)
 - | └─ crypto (owner-only: CA keys, server certs, signing keys)
- └─ services
 - | └─ http
 - | | └─ config (listen port, TLS cert ref, timeouts)
 - | | └─ routes (Prolog routing rules)
 - | | └─ static (file path mappings)
 - | | └─ sessions (session KB per active user)
 - | └─ email
 - | | └─ config (domain, relay settings, spam thresholds)
 - | | └─ users (child KB per user, each with inbox/sent/drafts)
 - | | └─ lists (mailing list KBs with subscriber facts)
 - | | └─ spam_rules (Prolog content analysis rules)
 - | | └─ queues
 - | | └─ outbound (outgoing message queue)
 - | | └─ moderation (held messages awaiting review)
 - | └─ dns
 - | | └─ config (recursion settings, forwarders)
 - | | └─ zones (child KB per zone with record facts)
 - | | └─ cache (LRU with TTL-based expiration)
 - | └─ auth

- | | └─ oauth (client registrations, token KBs)
- | | └─ saml (IdP config, signing keys)
- | └─ └─ ldap (directory tree mirroring org structure)
- | └─ mqtt
- | | └─ config (max connections, QoS defaults)
- | | └─ topics (child KB per topic hierarchy)
- | └─ chat
- | | └─ irc (channel KBs, user KBs)
- | | └─ xmpp (roster KBs, room KBs)
- | | └─ matrix (room KBs, federation config)
- | └─ storage
- | | └─ s3 (bucket KBs with object children)
- | | └─ ftp (home directory mappings)
- | | └─ webdav (resource tree)
- | └─ monitoring
- | | └─ metrics (per-service metric facts)
- | | └─ syslog (per-host log KBs)
- | | └─ snmp (MIB tree)
- | | └─ tracing (trace KBs with span facts)
- | └─ └─ alerts (alert rule KBs, notification queues)
- | └─ cache
- | | └─ redis (keyspace KBs)
- | └─ streaming

- | | └─ kafka (topic KBs with partition children)
- | | └─ hls (channel KBs with segment facts)
- | └─ calendar
 - | └─ caldav (per-user calendar KBs)
- | └─ voip
 - | └─ sip (registration KBs, call session KBs)
 - | └─ webrtc (signaling session KBs)
- | └─ print
 - | └─ ipp (printer KBs, job queue KBs)
- └─ org
 - └─ acme
 - └─ engineering (department KB with inherited constraints)
 - └─ users (child KB per user: credentials, grants, preferences)
 - └─ projects (child KB per project)
- └─ snapshots
 - └─ http_server_v3
 - └─ smtp_handler_v2
 - └─ dns_server_v1
 - └─ imap_handler_v2
 - └─ mqtt_broker_v1
 - └─ monitoring_poller_v4
 - └─ auth_server_v2
- └─ sessions

|— sess_4827 (active HTTP clone)

|— sess_4828 (active IMAP clone)

|— sess_4829 (active SSH clone)

Appendix D — Service Development Time Estimates

Service	Protocol complexity	Processing complexity	Grammar development	Rule development	Testing	Total estimate	Conventional equivalent
Static HTTP server	Low	Minimal	1 hour	1 hour	1 hour	3 hours	1-2 weeks (Nginx config + deployment)
JSON REST API	Low	Medium	1 hour	4-8 hours	2-4 hours	7-13 hours	4-8 weeks
SMTP inbound	Medium	Medium (spam)	2 hours	4-6 hours	2-3 hours	8-11 hours	4-8 weeks (Postfix + SpamAssassin)
IMAP server	Medium	Low	2 hours	3-5 hours	2-3 hours	7-10 hours	6-12 weeks
DNS authoritative	Low	Low	1 hour	1-2 hours	1-2 hours	3-5 hours	1-2 weeks (BIND config)
DNS recursive	Low	Medium	1 hour	2-4 hours	2-3 hours	5-8 hours	2-4 weeks
MQTT broker	Low	Low	1 hour	2-3 hours	1-2 hours	4-6 hours	2-4 weeks
IRC server	Low	Low	1 hour	2-3 hours	1-2 hours	4-6 hours	4-8 weeks
Redis-compatible cache	Low	Low	1 hour	2-3 hours	1-2 hours	4-6 hours	N/A (use Redis)
OAuth/OIDC provider	Medium	Medium	2 hours	4-6 hours	3-4 hours	9-12 hours	8-16 weeks

Service	Protocol complexity	Processing complexity	Grammar development	Rule development	Testing	Total estimate	Conventional equivalent
S3-compatible storage	Medium	Low	2 hours	3-5 hours	2-3 hours	7-10 hours	8-16 weeks (MinIO alternative)
Prometheus metrics	Low	Low	1 hour	1-2 hours	1 hour	3-4 hours	1-2 weeks
Full email stack	High	High	4 hours	10-16 hours	4-6 hours	18-26 hours	3-6 months
Full monitoring stack	Medium	Medium	3 hours	6-10 hours	3-5 hours	12-18 hours	2-4 months

Appendix E — Data Primitive Usage in Server Contexts

Primitive	Server use case	Typical sizing	Coordination role
Queue (FIFO)	SMTP outbound queue; task distribution; notification delivery; AMQP exchanges	100-10000 items	Producer-consumer decoupling
Counter	Active connections; requests per second; error count; bandwidth bytes; lease expiration; rate limiting; grant uses	Varies by metric	Threshold monitoring; pool management; rate control
LRU cache	DNS cache; session cache; parsed request cache; template cache; authentication token cache	100-100000 entries	TTL-based expiration; deduplication

Primitive	Server use case	Typical sizing	Coordination role
Lock	Batch fact assertion; DHCP address reservation; WebDAV resource locking; database transaction	One per resource	Cooperative signaling; consistency
Ring buffer	Per-minute request metrics; connection log; recent queries; error history	60-1440 entries	Rolling window; trend detection
Bitset	Endpoint health; feature flags; message delivery tracking; certificate status	64-4096 bits	Completion tracking; capability advertisement
Stack	Protocol state machine depth; nested transaction contexts; undo history	8-32 entries	State management; backtracking

Appendix F — Token Cost Comparison: Conventional vs LM Software Server

Operation	Conventional server	LM Software server	Savings
Parse HTTP request	0 (compiled code)	0 (grammar)	Equal
Route request	0 (compiled code)	8 (rule invocation) or 0 (auto-fire)	8 or 0 tokens
Authenticate user	0 (compiled code)	0 (Prolog rule auto-fire)	Equal
Query database	0 (compiled code)	8 (builtin invocation) or 0 (in rule chain)	8 or 0 tokens
Format JSON response	0 (compiled code)	0 (grammar)	Equal
Send response	0 (compiled code)	0 (granted primitive)	Equal
Log request	0 (compiled code)	0 (automatic provenance)	Equal
Handle novel request type	N/A (requires code change)	30-100 (LLM judgment)	N/A (capability difference)

Operation	Conventional server	LM Software server	Savings
Total typical API request	0 tokens (compiled)	8-24 tokens (level 2-3)	Tradeoff: tokens vs development time

Appendix G — Security Model Comparison

Security concern	Conventional server	LM Software server	Structural advantage
Authentication	Middleware module (PAM, OAuth library)	User KB credential facts + Prolog rule	No middleware to misconfigure
Authorization	ACL files, RBAC code, policy engine	Grant facts + scope chain	Two-integer check; no policy language
Rate limiting	Rate limiter middleware or proxy	Counter with threshold	Exact VDR fraction comparison
Input validation	Schema validation library	Grammar + constraint	Grammar rejects malformed structurally
SQL injection	Parameterized queries, ORM	N/A (no SQL engine; Prolog queries are typed)	Attack vector does not exist
Cross-site scripting	Output encoding library	Grammar produces safe output by construction	Structural output safety
Session fixation	Session management library	Clone-per-session with unique KB path	Each session is unique KB branch
CSRF	Token validation middleware	Grant system on operational primitives	Structural operation authorization
Data leakage between users	Application-level isolation	Sibling scope isolation	Structural impossibility
Audit trail	Logging framework, log management	Automatic provenance on every operation	Cannot be disabled; axiom-protected
Certificate management	Certbot, manual renewal	Fact in security KB; expiration counter; polling runner auto-renewal	Integrated lifecycle
Privilege escalation	Careful code review	Monotonic grants; no self-elevation	Structural prevention

Appendix H — Protocol State Machines as Prolog

Protocol	States	Transitions as Prolog rules	Example rule
SMTP	init, greeted, mail from, rcpt to, data, quit	state(Sess, init), received(Sess, ehlo(Domain)) → assert state(Sess, greeted), send(Sess, “250 OK”)	Greeting acceptance
FTP	init, authenticated, ready, transfer	state(Sess, init), received(Sess, user(Name)) → check user(Name), assert state(Sess, user given)	User identification
SIP	idle, trying, ringing, connected, terminated	state(Call, idle), received(Call, invite(SDP)) → assert state(Call, trying), send(Call, “100 Trying”)	Call initiation
IMAP	not auth, authenticated, selected, logout	state(Sess, authenticated), received(Sess, select(Mbox)) → load mailbox(Mbox), assert state(Sess, selected)	Mailbox selection
MQTT	connecting, connected, disconnecting	state(Sess, connecting), received(Sess, connect(ClientID)) → auth(ClientID), assert state(Sess, connected)	Client connection
DHCP	init, selecting, requesting, bound, renewing	state(Client, init), received(Client, discover(MAC)) → find address(Addr), offer(Client, Addr)	Address discovery

Protocol	States	Transitions as Prolog rules	Example rule
HTTP/2	idle, open, half closed local, half closed remote, closed	state(Stream, idle), received(Stream, headers(H)) → process headers(H), assert state(Stream, open)	Stream opening
SSH	transport, userauth, connection	state(Sess, transport), received(Sess, kexinit(Params)) → negotiate(Params), assert state(Sess, key exchange)	Key exchange
WebSocket	connecting, open, closing, closed	state(Sess, open), received(Sess, close(Code)) → send(Sess, close(Code)), assert state(Sess, closing)	Close handshake

Appendix I — Scaling Characteristics

Metric	Per-clone overhead	Scaling behavior	Bottleneck	Mitigation
Memory per clone	10-500 KB live state	Linear with connection count	Total system memory	Drift thresholds limit clone lifetime
CPU per clone at level 3	Prolog evaluation only	Linear with request rate	Prolog evaluation throughput	Rule optimization; predicate indexing
CPU per clone at level 1	Full LLM forward pass	Proportional to token count	LLM throughput	Rule accumulation moves work to level 3
Shared KB read access	Zero marginal cost	Constant regardless of clone count	None (read-only, no contention)	N/A

Metric	Per-clone overhead	Scaling behavior	Bottleneck	Mitigation
Queue throughput	One atomic op per push/pop	Limited by hardware atomics	~100M ops/sec per queue	Multiple queues for partitioned work
Counter throughput	One atomic op per mutation	Limited by hardware atomics	~100M ops/sec per counter	Per-clone local counters with periodic merge
Clone spawn time	Snapshot copy (10-500 KB)	Sub-millisecond per clone	Memory bandwidth	Snapshot size optimization
Clone kill time	Live state deallocation	Sub-millisecond per clone	None significant	N/A

Appendix J — Grant Requirements by Service Category

Service category	Network listen	Network send	Filesystem	Crypto	Process	Database
HTTP server	Required	Optional (proxy)	Optional (static)	Required (TLS)	No	Optional
Email (SMTP)	Required	Required (relay)	Optional (storage)	Required (DKIM/TLS)	No	No
Email (IMAP)	Required	No	No	Required (TLS)	No	No
DNS	Required	Required (recursion)	No	Optional (DNSSEC)	No	No
SSH	Required	No	Required (shell)	Required (keys)	Optional (exec)	No
FTP	Required	No	Required	Optional (TLS)	No	No
MQTT	Required	Optional (bridge)	No	Optional (TLS)	No	No
Auth (OAuth)	Required	No	No	Required (signing)	No	No
Cache (Redis)	Required	No	No	Optional (TLS)	No	No
Monitoring	Required	Optional (alerts)	No	Optional (TLS)	No	No
Storage (S3)	Required	No	Required	Required (TLS)	No	No

Service category	Network listen	Network send	Filesystem	Crypto	Process	Database
VoIP (SIP)	Required	Required (calls)	No	Optional (TLS)	No	No

Appendix K — Migration Path from Conventional Services

Current service	Migration approach	Coexistence strategy	Data migration	Cutover risk
Apache/Nginx HTTP	Develop LM Software HTTP handler; proxy traffic during testing	Run behind existing reverse proxy; route percentage of traffic to LM Software handler	Static files: reference same filesystem paths; dynamic: port API endpoints incrementally	Low: per-endpoint migration
Postfix/Sendmail SMTP	Develop LM Software SMTP handler; relay through existing MTA during testing	Configure as secondary MX; gradually shift priority	Import existing mailboxes as KB facts with provenance	Medium: email is critical path
BIND DNS	Develop LM Software DNS server; load zone files as KB facts	Run as secondary nameserver; monitor response consistency	Zone files parse directly to KB facts	Low: secondary NS is standard
OpenLDAP	Develop LM Software LDAP server; sync directory tree as KB facts	Run parallel LDAP servers; sync with polling runner	LDIF export imports as KB facts	Medium: authentication dependency
Redis/Memcached	Develop LM Software cache; dual-write during migration	Application writes to both; reads from LM Software with fallback	Dump import as KB facts	Low: cache can be cold-started

Current service	Migration approach	Coexistence strategy	Data migration	Cutover risk
Mosquitto MQTT	Develop LM Software MQTT broker; bridge during migration	MQTT bridge between old and new brokers	Topic structure maps to KB tree directly	Low: MQTT designed for broker migration
Prometheus	Develop LM Software metrics endpoint; scrape both during testing	Prometheus scrapes both endpoints; compare results	Historical data not migrated; forward-only	Low: monitoring is supplementary

Appendix L — Binary Grammar Field Extraction Patterns

Protocol	Field	Byte position	Bit width	VDR representation	Extraction method
DNS header	ID	0-1	16	Integer fact	Big-endian u16
DNS header	QR flag	2	1 (bit 7)	Boolean fact	Bit mask
DNS header	Opcode	2	4 (bits 3-6)	Integer fact	Bit shift + mask
DNS header	QDCOUNT	4-5	16	Integer fact	Big-endian u16
DNS question	QNAME	Variable	Label-length encoded	Interned string ID	Label walk with pointer compression
MQTT fixed header	Packet type	0	4 (bits 4-7)	Integer fact	Bit shift
MQTT fixed header	Remaining length	1+	7-28 (variable)	Integer fact	Variable-byte decode loop
NTP	Leap indicator	0	2 (bits 6-7)	Integer fact	Bit mask

Protocol	Field	Byte position	Bit width	VDR representation	Extraction method
NTP	Transmit timestamp	40-47	64	VDR fraction: integer part / 1, fractional part / 2^{32}	Big-endian u64 split
RADIUS	Code	0	8	Integer fact	Direct byte
RADIUS	Length	2-3	16	Integer fact	Big-endian u16
RADIUS	Authenticator	4-19	128	Byte buffer fact	Raw 16 bytes
SNMP (BER)	Tag	0	8	Integer fact	Tag-class + constructed + number
SNMP (BER)	Length	1+	7-variable	Integer fact	Short form (1 byte) or long form (multi-byte)
RTP	Version	0	2 (bits 6-7)	Integer fact	Bit mask
RTP	Payload type	1	7 (bits 0-6)	Integer fact	Bit mask
RTP	Sequence number	2-3	16	Integer counter fact	Big-endian u16
RTP	Timestamp	4-7	32	Integer fact	Big-endian u32
RTP	SSRC	8-11	32	Integer fact	Big-endian u32
SSH	Packet length	0-3	32	Integer fact	Big-endian u32
SSH	Padding length	4	8	Integer fact	Direct byte
SSH	Message type	5	8	Integer fact	Direct byte
HTTP/2	Frame length	0-2	24	Integer fact	Big-endian u24
HTTP/2	Type	3	8	Integer fact	Direct byte
HTTP/2	Flags	4	8	Bitset fact	8-bit flags
HTTP/2	Stream ID	5-8	31 (bit 0 reserved)	Integer fact	Big-endian u32 masked
DHCP	Op	0	8	Integer fact (1=request, 2=reply)	Direct byte

Protocol	Field	Byte position	Bit width	VDR representation	Extraction method
DHCP	XID	4-7	32	Integer fact	Big-endian u32
DHCP	CIADDR	12-15	32	Dotted-quad string via grammar	4 bytes formatted
DHCP	CHADDR	28-33	48	MAC address string via grammar	6 bytes formatted
CoAP	Version	0	2 (bits 6-7)	Integer fact	Bit mask
CoAP	Type	0	2 (bits 4-5)	Integer fact	Bit mask
CoAP	Token length	0	4 (bits 0-3)	Integer fact	Bit mask
CoAP	Code	1	8	Integer fact (class.detail)	Split 3+5 bits
AMQP	Frame type	0	8	Integer fact	Direct byte (method/header/body/heartbeat)
AMQP	Channel	1-2	16	Integer fact	Big-endian u16
AMQP	Frame size	3-6	32	Integer fact	Big-endian u32
IPP	Version	0-1	16	Integer fact (major.minor)	Two bytes
IPP	Operation ID	2-3	16	Integer fact	Big-endian u16
IPP	Request ID	4-7	32	Integer fact	Big-endian u32

Appendix N — Cryptographic Primitive Usage by Service

Service	Operation	Primitive	Key storage	Provenance
HTTPS/TLS	Server key exchange	crypto dh or crypto ecdh	Server private key: owner-only KB	Key generation timestamp, algorithm
HTTPS/TLS	Certificate presentation	crypto sign verify	Certificate chain: server security KB	Issuer, validity period, serial

Service	Operation	Primitive	Key storage	Provenance
HTTPS/TLS	Symmetric encryption	crypto aes gcm	Session key: session KB (live state)	Negotiated cipher suite
SSH	Host key verification	crypto ed25519 verify	Host key: server security KB	Key fingerprint, first-seen timestamp
SSH	User authentication	crypto ed25519 verify	User public key: user KB authorized keys	Key comment, added-by, added-when
SMTP DKIM	Message signing	crypto rsa sign	DKIM private key: domain security KB	Selector, domain, algorithm
SMTP DKIM	Signature verification	crypto rsa verify	Sender public key: fetched via DNS TXT	DNS lookup timestamp, key source
OAuth	Token signing (JWT)	crypto hmac or crypto rsa sign	Signing key: auth server owner-only KB	Key rotation timestamp, algorithm
SAML	Assertion signing	crypto rsa sign or crypto ecdsa sign	IdP signing key: owner-only KB	Certificate serial, validity
Matrix federation	Event signing	crypto ed25519 sign	Server signing key: owner-only KB	Key ID, valid until timestamp
Matrix E2EE	Olm session	crypto curve25519 + crypto aes	Device keys: user device KB	Device ID, creation timestamp
DNSSEC	Zone signing	crypto rsa sign or crypto ecdsa sign	ZSK/KSK: zone security KB	Key tag, algorithm, inception, expiration
OCSP	Response signing	crypto rsa sign	OCSP responder key: CA security KB	Responder ID, produced-at timestamp
Certificate authority	CSR signing	crypto rsa sign or crypto ecdsa sign	CA private key: root security KB, owner-only	Serial number, validity period, issuer

Service	Operation	Primitive	Key storage	Provenance
RADIUS	Authenticator	crypto md5 (legacy)	Shared secret: RADIUS config KB	Per-request authenticator
SNMP v3	Authentication	crypto hmac sha	USM user key: SNMP security KB	Engine ID, boots, time
SNMP v3	Privacy	crypto aes cfb	USM privacy key: SNMP security KB	Engine ID, boots, time
WebRTC DTLS	Media encryption	crypto dtls srtp	DTLS certificate: session KB	Fingerprint from SDP
ActivityPub	HTTP signatures	crypto rsa sign	Actor private key: actor security KB	Key ID URL, created timestamp
S3 presigned URL	Request signing	crypto hmac sha256	AWS-compatible secret: storage security KB	Expiration timestamp, scope
IPP/IPPS	Channel encryption	crypto tls	Printer certificate: print security KB	Printer identity, validity

Appendix O — Rate Limiting Patterns by Service

Service	Rate limit type	Counter path	Threshold	Window	Action on breach
HTTP API	Requests per minute per user	root.services.http.limits.user.id.rpm	60	1 minute (ring buffer)	429 Too Many Requests
HTTP API	Requests per second global	root.services.http.limits.global.rps	10000	1 second (counter reset by poller)	503 Service Unavailable
SMTP	Messages per hour per sender	root.services.smtp.limits.sender.mph	100	1 hour (ring buffer)	421 Try again later
SMTP	Recipients per message	root.services.smtp.limits.session.mrcpts	100	Per message	452 Too many recipients
SMTP	Connection rate per IP	root.services.smtp.limits.ip.cph	100	1 minute	Connection refused

Service	Rate limit type	Counter path	Threshold	Window	Action on breach
DNS	Queries per second per IP	root.services.dns	5000	limits.ip.qps 1 second	Drop query (UDP)
MQTT	Publishes per minute per client	root.services.mqtt	1000	limits.client.ppm 1 minute	DISCONNECT
MQTT	Subscriptions per client	root.services.mqtt	50	limits.client.subs lifetime	SUBACK with error
IRC	Messages per second per user	root.services.irc	20	limits.user.mps 1 second	Temporary mute
SSH	Auth attempts per connection	root.services.ssh	3	limits.session.auths session	Disconnect
SSH	Connections per minute per IP	root.services.ssh	5	limits.ip.cpm 1 minute	Connection refused
FTP	Failed logins per IP	root.services.ftp	5	limits.ip.fails 10 minutes	IP blocked (temporary fact)
LDAP	Searches per minute per bind DN	root.services.ldap	30	limits.dn.spm 1 minute	Return sizeLimitExceeded
OAuth	Token requests per minute per client	root.services.oauth.id.tpm	60	limits.client 1 minute	429 response
Redis	Commands per second per client	root.services.cache	10000	limits.client.cps 1 second	Slow down response
SIP	INVITE per minute per caller	root.services.voice	10	limits.caller.ipm 1 minute	486 Busy Here
GraphQL	Query complexity per request	root.services.http	10000	limits.user.complexity 1 request	400 Query Too Complex
WebSocket	Frames per second per connection	root.services.http	100	limits.session.fps 1 second	Close connection

Service	Rate limit type	Counter path	Threshold	Window	Action on breach
S3	PUT requests per minute per bucket	root.services.storage.limits.bucket.put	3500	1 minute	503 SlowDown
Syslog	Messages per second per source	root.services.monitoring.limits.messages	500	1 second	Drop messages

Appendix P — Health Check Patterns for Polling Runners

Service	Health metric	Check method	Healthy threshold	Warning threshold	Critical threshold	Polling interval
HTTP	Response time	Timer around test request	< 200ms	200-500ms	> 500ms	30 seconds
HTTP	Active connections	Counter read	< 80% of max	80-95%	> 95%	15 seconds
HTTP	Error rate (5xx)	Counter ratio: errors/total	< 1/100	1/100 - 5/100	> 5/100	30 seconds
SMTP	Queue depth	Queue length read	< 100	100-500	> 500	60 seconds
SMTP	Delivery latency	Timestamp diff on recent deliveries	< 30 seconds	30-120 seconds	> 120 seconds	60 seconds
DNS	Query latency	Timer around test query	< 5ms	5-20ms	> 20ms	15 seconds
DNS	Cache hit rate	Counter ratio: hits/total	> 80/100	60/100 - 80/100	< 60/100	60 seconds
IMAP	Active sessions	Counter read	< 80% of max	80-95%	> 95%	30 seconds
MQTT	Connected clients	Counter read	< 90% of max	90-98%	> 98%	30 seconds

Service	Health metric	Check method	Healthy threshold	Warning threshold	Critical threshold	Polling interval
MQTT	Message backlog	Queue depth sum across topics	< 1000	1000-5000	> 5000	30 seconds
Redis	Memory usage	Counter read	< 70% of max	70-90%	> 90%	15 seconds
Redis	Eviction rate	Counter delta per interval	< 10/minute	10-100/minute	> 100/minute	30 seconds
SSH	Failed auth rate	Counter ratio per interval	< 5/100	5/100 - 20/100	> 20/100	60 seconds
LDAP	Bind failures	Counter per interval	< 10/hour	10-50/hour	> 50/hour	60 seconds
DHCP	Pool utilization	Leased/total ratio	< 80/100	80/100 - 95/100	> 95/100	300 seconds
S3	Storage utilization	Sum of object sizes	< 80% of quota	80-95%	> 95%	600 seconds
Syslog	Ingestion rate	Counter delta per interval	Within 2 × baseline	2-5 × baseline	> 5 × baseline	30 seconds
Kafka	Consumer lag	Offset diff: latest - committed	< 1000	1000-10000	> 10000	15 seconds
Prometheus	Scrape duration	Timer around scrape cycle	< 5 seconds	5-15 seconds	> 15 seconds	60 seconds
NTP	Stratum	Fact read	≤ 3	4-6	> 6 or 16 (unsync)	300 seconds
CA	Certificate expiration	Timestamp diff on all certs	> 30 days	7-30 days	< 7 days	3600 seconds

Appendix Q — Session State Machine Transitions

Protocol	From state	Event	Guard condition (Prolog)	To state	Side effects
SMTP	init	EHLO received	valid domain(Domain)	greeted	Assert capabilities; send 250
SMTP	greeted	MAIL FROM received	not rate limited(Sender)	mail from	Assert envelope sender; send 250
SMTP	mail from	RCPT TO received	user exists(Rcpt), rcpt count(N), $N < \text{max rcpts}$	rcpt to	Assert recipient; increment rcpt counter; send 250
SMTP	rcpt to	DATA received	rcpt count(N), $N \geq 1$	data	Send 354
SMTP	data	Content received	message size(S), $S \leq \text{max size}$	completed	Parse message; run spam cascade; deliver; send 250
SMTP	any	QUIT received	true	quit	Send 221; close connection
SMTP	any	RSET received	true	greeted	Retract envelope facts; send 250
IMAP	not auth	LOGIN received	valid credentials(User, Pass)	authenticated	Load user grants; send OK
IMAP	authenticated	SELECT received	mailbox exists(Mbox), has grant(read, Mbox)	selected	Set active mailbox; send flags, exists, recent, OK

Protocol	From state	Event	Guard condition (Prolog)	To state	Side effects
IMAP	selected	FETCH received	valid sequence(Seq)	selected	Query message facts; fill FETCH response grammar
IMAP	selected	STORE received	has grant(write, Mbox)	selected	Modify flag facts; send FETCH update
IMAP	selected	CLOSE received	true	authenticated	Expunge deleted; clear active mailbox
IMAP	any	LOGOUT received	true	logout	Send BYE, OK; close connection
FTP	init	USER received	true	user given	Store username; send 331
FTP	user given	PASS received	valid credentials(User, Pass)	authenticated	Load grants; send 230
FTP	authenticated	CWD received	dir exists(Path), has grant(read, Path)	authenticated	Update working directory binding; send 250
FTP	authenticated	PASV received	true	passive wait	Open data port; send 227 with port
FTP	passive wait	Data connection established	true	authenticated	Ready for transfer
FTP	authenticated	RETR received	file exists(Path), has grant(read, Path)	transferring	Stream file content; send 226 on complete

Protocol	From state	Event	Guard condition (Prolog)	To state	Side effects
FTP	authenticated	STOR received	has grant(write, Path)	transferring	Receive file content; write via primitive; send 226
SIP	idle	INVITE received	call authorized(Caller, Callee)	trying	Assert call facts; send 100 Trying; forward to callee
SIP	trying	180 Ringing from callee	true	ringing	Forward 180 to caller
SIP	ringing	200 OK from callee	true	connected	Forward 200 to caller; establish media path
SIP	connected	BYE from either	true	terminated	Send 200 OK; release media; log CDR facts
MQTT	connecting	CONNECT received	auth ok(ClientID, User, Pass)	connected	Assert client session facts; send CON-NACK
MQTT	connected	SUBSCRIBE received	topic authorized(Client, Topic)	connected	Assert subscription connection; send SUBACK
MQTT	connected	PUBLISH received (QoS 0)	topic authorized(Client, Topic)	connected	Assert message fact; distribute to subscribers

Protocol	From state	Event	Guard condition (Prolog)	To state	Side effects
MQTT	connected	PUBLISH received (QoS 1)	topic authorized(Client, Topic)	connected	Assert message; distribute; send PUBACK
MQTT	connected	DISCONNECT received	True	disconnected	Clean session if clean session flag; process LWT if unclean
SSH	transport	KEXINIT received	algorithms compatible(Proposed)	key exchange	Negotiate algorithms; perform DH exchange
SSH	key exchange	NEWKEYS received	key derived(SessionKey)	authenticated	Switch to encrypted channel
SSH	authenticated	USERAUTH REQUEST	method supported(Method)	authenticating	Check credentials via method
SSH	authenticating	Auth success	valid credentials(User)	connection	Send USERAUTH SUCCESS; load user grants
SSH	connection	CHANNEL OPEN	channel type allowed(Type)	connection	Allocate channel; send CHANNEL OPEN CONFIRMATION
HTTP/2	idle (stream)	HEADERS received	stream id valid(ID), max streams ok	open	Parse headers; route to handler

Protocol	From state	Event	Guard condition (Prolog)	To state	Side effects
HTTP/2	open	DATA received	flow window > 0	open	Buffer body data; decrement flow window
HTTP/2	open	Response complete	true	half closed local	Send HEADERS + DATA frames; send END STREAM
HTTP/2	half closed local	RST STREAM received	true	closed	Release stream resources
WebSocket	connecting	Upgrade accepted	valid hand-shake(Key)	open	Send 101 Switching Protocols with Accept header
WebSocket	open	Text frame received	true	open	Parse payload as fact; process via rules
WebSocket	open	Ping received	true	open	Send Pong with same payload
WebSocket	open	Close received	true	closing	Send Close frame; set close timer
WebSocket	closing	Close sent + received	true	closed	Release resources
DHCP	init	DISCOVER received	pool has available(Pool)	offering	Select address; set lock; send OFFER
DHCP	offering	REQUEST received	offered to client(Addr, MAC)	bound	Release lock; assert lease fact with TTL counter; send ACK

Protocol	From state	Event	Guard condition (Prolog)	To state	Side effects
DHCP	bound	Lease expires (counter)	lease counter(Addr) == 0	init	Retract lease; return address to pool
DHCP	bound	RELEASE received	lease matches(Addr, MAC)	init	Retract lease; return address to pool

Appendix R — Error Response Grammar Templates by Protocol

Protocol	Error class	Grammar template	Content slots	HTTP equivalent
SMTP	Temporary failure	“4xx text”	sub-code, description	5xx
SMTP	Permanent failure	“5xx text”	sub-code, description	4xx
SMTP	Auth required	“530 Authentication required”	None (fixed)	401
IMAP	Command failed	“tag NO text”	tag, description	403/404
IMAP	Bad syntax	“tag BAD text”	tag, description	400
FTP	Not logged in	“530 Not logged in”	None (fixed)	401
FTP	File not found	“550 path: No such file”	path	404
FTP	Permission denied	“550 path: Permission denied”	path	403
DNS	NXDOMAIN	Response with RCODE=3	Query echoed, no answers	404
DNS	SERVFAIL	Response with RCODE=2	Query echoed, no answers	500
DNS	REFUSED	Response with RCODE=5	Query echoed, no answers	403

Protocol	Error class	Grammar template	Content slots	HTTP equivalent
MQTT	Connection refused	CONNACK with return code 1-5	Return code integer	401/403
MQTT	Not authorized	SUBACK with failure code	Topic, failure reason	403
SSH	Auth failure	USERAUTH FAILURE	Allowed methods list	401
SSH	Channel failure	CHANNEL OPEN FAILURE	Reason code, description	403
HTTP	Bad request	“HTTP/1.1 400 Bad Requestheadersjson error”	headers, error detail	400
HTTP	Unauthorized	“HTTP/1.1 401 UnauthorizedWWW-Authenticate: scheme”	scheme	401
HTTP	Forbidden	“HTTP/1.1 403 Forbiddenheadersjson error”	headers, error detail	403
HTTP	Not found	“HTTP/1.1 404 Not Foundheadersjson error”	headers, error detail	404
HTTP	Rate limited	“HTTP/1.1 429 Too Many RequestsRetry-After: seconds”	seconds	429
HTTP	Server error	“HTTP/1.1 500 Internal Server Errorheadersjson error”	headers, error detail	500
GraphQL	Parse error	“{“errors”:[{“message”: <i>msg</i> ,”locations”:[{“line”: <i>l</i> ,”column”: <i>c</i> }]}”		400
GraphQL	Validation error	“{“errors”:[{“message”: <i>msg</i> ,”path”:[<i>path</i>]}]}”		400

Protocol	Error class	Grammar template	Content slots	HTTP equivalent
gRPC	Status	Trailer frame: “grpc-status: codegrpc- message: msg”	code (0-16), message	Varies by code
Redis RESP	Error	“-type message”	error type, message	Varies
LDAP	Error result	Result message with resultCode and diagnosticMes- sage	code (0-80), message	Varies
SIP	Client error	“method SIP/2.0” or “SIP/2.0 code reason”	code (400-499), reason	4xx
SIP	Server error	“SIP/2.0 code reason”	code (500-599), reason	5xx
RADIUS	Access reject	Access-Reject packet with Reply- Message	rejection reason	403
SNMP	Error response	Response PDU with error-status and error-index	status (1-18), index	Varies
S3	Error response	XML: “codemsgid”	code, message, request ID	Varies
WebSocket	Close frame	Opcode 8, status code (2 bytes), reason text	code (1000-4999), reason	N/A
AMQP	Channel close	channel.close method with reply-code and reply-text	code, text, method-id, class-id	Varies
CoAP	Error response	4.xx or 5.xx response code with diagnostic payload	code, diagnostic	4xx/5xx

Appendix S — Cross-Service Data Flow Patterns

Pattern	Source service	Target service	Data path	Trigger	Example
Auth chain	HTTP → OAuth → LDAP	HTTP request → OAuth token validation → LDAP user lookup	Token fact → scope facts → user attribute facts	HTTP request with Bearer token	API authenticating user and loading permissions
Email delivery	SMTP → DNS → SMTP	Inbound message → MX lookup → outbound relay	Message fact → MX record query → outbound queue push	Inbound SMTP DATA completion	Email forwarding/relaying
Certificate lifecycle	CA → HTTP → monitoring	CSR signing → certificate deployment → expiration monitoring	Certificate fact → server security KB → expiration counter	CSR submission	Automated certificate provisioning
Log aggregation	Syslog → Prometheus → HTTP	Log ingestion → metric derivation → dashboard query	Log facts → counter increments → metric exposition → HTML template	Log message arrival	Error rate dashboard
IoT data pipeline	MQTT → Kafka → HTTP	Sensor publish → stream storage → API query	Topic fact → partition fact → query response	MQTT PUBLISH	Sensor data API
User provisioning	LDAP → IMAP → FTP → SSH	Directory entry → mailbox creation → home directory → SSH key	User fact → inbox KB → filesystem grant → authorized keys fact	LDAP ADD operation	New employee onboarding

Pattern	Source service	Target service	Data path	Trigger	Example
Alerting chain	Prometheus → SMTP → XMPP	Threshold breach → email notification → IM notification	Alert fact → outbound email queue → XMPP message stanza	Metric exceeds threshold	Multi-channel alerting
Federation sync	ActivityPub → Matrix	Mastodon post → Matrix room message	Activity fact → room event fact	Inbound federation POST	Cross-platform messaging
Backup verification	S3 → monitoring → SMTP	Object checksum → integrity check → notification	Checksum fact → comparison → alert queue → email	Scheduled integrity scan	Backup integrity monitoring
Single sign-on	HTTP → SAML → LDAP → HTTP	SP request → IdP redirect → auth → assertion → SP redirect	Request fact → assertion facts → user grants → session KB	Unauthenticated HTTP request	SSO login flow
Calendar scheduling	CalDAV → SMTP → CalDAV	Meeting invite → email delivery → attendee calendar update	VEVENT fact → email with iCalendar attachment → recipient calendar fact	Meeting organizer creates event	Meeting invitation workflow
Compliance logging	Any service → Syslog → S3	Operation audit → centralized log → long-term archive	Provenance fact → syslog message → S3 object	Any auditable operation	Regulatory compliance archival

Appendix T — VDR Fraction Usage in Server Contexts

Context	Value	VDR representation	Conventional representation	Advantage
Financial transaction	\$10,000.00	[1000000, 100, 0]	10000.0 (float64)	No rounding; no cent ambiguity
Tax rate	8.875%	[8875, 100000, 0]	0.08875 (float64)	Exact multiplication with amount
SLA uptime	99.99%	[9999, 10000, 0]	0.9999 (float64)	Exact threshold comparison
Bandwidth rate	2.5 Gbps	[25, 10, 0]	2.5 (float64)	Exact rate calculation
Token bucket refill	16.67 tokens/sec	[1667, 100, 0]	16.666666... (float64)	No accumulation drift
Disk usage	73.2%	[732, 1000, 0]	0.732 (float64)	Exact threshold comparison
Error rate	0.03%	[3, 10000, 0]	0.0003 (float64)	No false threshold crossings
Request latency p99	142.7 ms	[1427, 10, 0] ms	142.7 (float64)	Exact percentile computation
Compression ratio	83.4%	[834, 1000, 0]	0.834 (float64)	Exact savings calculation
QoS weight	1/3 of bandwidth	[1, 3, 0]	0.33333... (float64)	Exact fair-share; sums to exactly 1
DHCP lease duration	86400 seconds	[86400, 1, 0]	86400 (int or float)	Exact expiration calculation
DNS TTL remaining	2847.5 seconds	[28475, 10, 0]	2847.5 (float64)	Exact cache expiration
Jitter measurement	3.7 ms	[37, 10, 0]	0.0037 (float64)	Exact buffer sizing
Packet loss rate	0.12%	[12, 10000, 0]	0.0012 (float64)	Exact QoS comparison
Load balance weight	3:2:1 ratio	[3,6,0], [2,6,0], [1,6,0]	0.5, 0.333..., 0.166...	Sums to exactly 1; no residual
OAuth token TTL	3600 seconds	[3600, 1, 0]	3600 (int)	Exact expiration

Context	Value	VDR representation	Conventional representation	Advantage
Connection timeout	30.5 seconds	[305, 10, 0]	30.5 (float64)	Exact timeout comparison
Rate limit refill	1.67/second	[167, 100, 0]	1.666... (float64)	No token drift over time
Video segment duration	6.006 seconds	[6006, 1000, 0]	6.006 (float64)	Exact HLS manifest timing
Audio sample rate	44100 Hz	[44100, 1, 0]	44100 (int)	Exact timing calculation

Appendix U — Concurrent Access Patterns on Shared KBs

KB type	Readers	Writers	Contention mechanism	Consistency guarantee	Example
DNS zone	Many (query clones)	Few (admin updates)	Read: no lock needed; Write: lock during batch update	Readers see complete zone or previous version	Zone record query during zone update
User credentials	Many (auth clones)	Rare (password changes)	Read: no lock; Write: retract + assert (atomic pair)	Auth uses fact present at query time	Login during password change
Email inbox	One (IMAP session)	One (SMTP delivery)	SMTP: assert new fact; IMAP: query existing facts	New messages appear on next FETCH	Email arriving during IMAP session
MQTT topic	Many (subscriber clones)	Many (publisher clones)	Publish: assert fact (append-only); Subscribe: query facts	Subscribers see messages asserted before their query	Multiple publishers, multiple subscribers

KB type	Readers	Writers	Contention mechanism	Consistency guarantee	Example
Rate limit counter	Many (request clones)	Many (same clones incrementing)	Atomic counter increment	Counter reflects all increments; may briefly lag	High-traffic rate limiting
Session store	One (owning clone)	One (same clone)	No contention: single owner	Complete consistency	Session data read/write
Certificate store	Many (TLS handshake clones)	Rare (cert rotation)	Read: no lock; Write: retract + assert	Clones use cert present at handshake start	Certificate renewal during active connections
Firewall rules	Many (packet filter clones)	Rare (admin updates)	Read: no lock; Write: lock during batch	Rules evaluated atomically: old set or new set	Rule update during traffic
Log storage	Few (query clones)	Many (ingestion clones)	Write: append-only (no contention); Read: query at point in time	Readers see consistent snapshot of ingested logs	Log query during high ingestion
Kafka partition	Few (consumer clones)	Many (producer clones)	Write: atomic counter increment for offset; Read: query by offset range	Consumers see committed offsets; producers append above	Concurrent produce and consume
DHCP pool	Few (request clones)	Same (lease assertions)	Lock per address during OFFER→ACK	Address offered to one client at a time	Simultaneous DISCOVER from multiple clients

KB type	Readers	Writers	Contention mechanism	Consistency guarantee	Example
OAuth token store	Many (validation clones)	Few (token issuance clones)	Write: assert new token fact; Read: query by token value	Token valid from assertion timestamp onward	Token validation during issuance burst
S3 bucket	Many (GET clones)	Many (PUT clones)	Write: assert new version fact; Read: query latest version	Readers see complete object or previous version	Concurrent upload and download

Appendix V — Protocol Keepalive and Timeout Management

Protocol	Keepalive mechanism	Timeout type	Counter/timer implementation	Action on timeout
HTTP/1.1	Keep-Alive header	Idle timeout	Counter: seconds since last request; poller checks	Close connection; kill clone
HTTP/2	PING frame	Idle + PING timeout	Counter: seconds since last frame; PING sent by rule	GOAWAY frame; close connection
SMTP	None (per-message)	Command timeout	Counter: seconds since last command	421 timeout; close
IMAP	IDLE command	Inactivity timeout	Counter: minutes since last command (excluding IDLE)	BYE; close
FTP	NOOP command	Idle timeout	Counter: seconds since last command	421 timeout; close
SSH	SSH MSG IGNORE	Keepalive interval	Counter: seconds since last packet; send keepalive	Disconnect after N missed responses

Protocol	Keepalive mechanism	Timeout type	Counter/timer implementation	Action on timeout
IRC	PING/PONG	Ping timeout	Counter: seconds since last PONG; PING sent by rule	Disconnect; remove from channels
XMPP	Whitespace ping or XEP-0199	Inactivity	Counter: seconds since last stanza	Close stream
MQTT	PINGREQ/PINGRESP	Keep-alive interval (client-set)	Counter: 1.5 × keep-alive since last packet	Disconnect; process LWT
WebSocket	Ping/Pong frames	Ping interval	Counter: seconds since last pong; ping sent by rule	Close connection
LDAP	Abandon stale ops	Idle timeout	Counter: seconds since last operation	Unbind; close
SIP	Session timer (UPDATE/re-INVITE)	Session expiration	Counter: seconds since last session refresh	BYE; release media
Redis	CLIENT SETNAME or PING	Idle timeout	Counter: seconds since last command	Close connection
AMQP	Heartbeat frames	Heartbeat timeout (negotiated)	Counter: seconds since last frame; heartbeat sent	Close connection
DNS/TCP	None (per-query)	Query timeout	Counter: seconds since connection open	Close TCP connection
Kafka	Heartbeat (consumer group)	Session timeout	Counter: seconds since last heartbeat	Rebalance; revoke partitions
gRPC	HTTP/2 PING	Keepalive interval	Counter: seconds since last frame	GOAWAY; close

Protocol	Keepalive mechanism	Timeout type	Counter/timer implementation	Action on timeout
CoAP	Confirmable retry	Retransmission timeout	Counter: doubling backoff from 2s to 32s	Give up after MAX RE-TRANSMIT (4)
Matrix	Sync long-polling	Sync timeout	Counter: seconds since last sync; configurable by client	Client reconnects; server cleans state

Appendix W — Service Composition Patterns

Composite service	Components	Wiring	KB tree layout	Clone interaction
Webmail	HTTP + IMAP + SMTP	HTTP clone reads inbox via IMAP KB facts; compose sends via SMTP outbox queue	root.services.http, with shared user KBs	HTTP clone reads email KBs directly; no IMAP protocol needed internally
Unified messaging	IRC + XMPP + Matrix + SMTP	Bridge clone subscribes to all protocols; cross-posts via queues	root.services.chat, with connection facts to each protocol	Bridge runner pops from each protocol's queue, pushes to others
CI/CD pipeline	HTTP (webhook) + SSH (build) + S3 (artifacts) + SMTP (notification)	Webhook clone triggers build via queue; build clone stores artifacts; notification on completion	root.services.http, with project KB linking them	Queue storage, hook→build→artifact→notification
Monitoring dashboard	Prometheus + Syslog + HTTP + SMTP	HTTP clone queries metric and log KBs; alert rules push to email queue	root.services.monitoring, with shared alert KB	HTTP clone reads monitoring KBs; alert poller pushes to email queue

Composite service	Components	Wiring	KB tree layout	Clone interaction
Identity platform	LDAP + OAuth + SAML + HTTP	HTTP consent page; LDAP backend auth; OAuth and SAML token generation	root.services.auth with shared user KB	Auth clones, saml query same user KB; token clones assert to protocol-specific token KBs
File sharing	S3 + HTTP + WebDAV + FTP	All protocols access same storage KBs; HTTP provides web UI	root.services.storage with protocol-specific handlers	All protocol clones read/write same object KBs with different grammars
IoT platform	MQTT + Kafka + HTTP + Prometheus	MQTT ingests sensor data; Kafka stores streams; HTTP serves API; Prometheus exposes metrics	root.services.mqtt with shared sensor KBs	MQTT, http, monitoring processor writes to Kafka topic KBs; HTTP clone reads them; metrics derived by internal runner
Collaboration suite	CalDAV + CardDAV + XMPP + WebDAV + SMTP	Shared user KBs; calendar invites via email; chat with presence	root.services.calendar with shared user KBs	Calendar clone, email sends invite via email queue; chat clone reads presence from user KB

Appendix X — Clone Memory Budget by Service

Service	Snapshot base	Per-request live state	Peak live state per clone	Typical clone lifetime	Memory × concurrent clones
Static HTTP	15 KB	2 KB (request facts)	17 KB	~50 ms	1000 clones × 17 KB = 17 MB
REST API	40 KB	5-20 KB (request + query results)	60 KB	~100 ms	1000 × 60 KB = 60 MB

Service	Snapshot base	Per-request live state	Peak live state per clone	Typical clone lifetime	Memory $\times$ concurrent clones
SMTP inbound	30 KB	10-500 KB (message body)	530 KB	~2 seconds	100×530 KB = 53 MB
IMAP session	50 KB	5-50 KB (selected mailbox cache)	100 KB	~30 minutes	500×100 KB = 50 MB
DNS query	20 KB	1 KB (query + answer)	21 KB	~5 ms	5000×21 KB = 105 MB
MQTT client	25 KB	2-10 KB (subscription state)	35 KB	~hours	10000×35 KB = 350 MB
IRC session	30 KB	5-20 KB (channel memberships)	50 KB	~hours	1000×50 KB = 50 MB
SSH shell	60 KB	10-50 KB (command history, state)	110 KB	~30 minutes	200×110 KB = 22 MB
FTP session	35 KB	5-20 KB (directory cache)	55 KB	~10 minutes	100×55 KB = 5.5 MB
Redis command	25 KB	1-5 KB (command + result)	30 KB	~1 ms	10000×30 KB = 300 MB
GraphQL query	45 KB	10-100 KB (resolved tree)	145 KB	~200 ms	500×145 KB = 72.5 MB
OAuth token request	35 KB	3 KB (token facts)	38 KB	~50 ms	500×38 KB = 19 MB
SIP call	40 KB	5-15 KB (call state, SDP)	55 KB	~minutes	200×55 KB = 11 MB
Syslog message	15 KB	1 KB (parsed message)	16 KB	~2 ms	2000×16 KB = 32 MB

Service	Snapshot base	Per-request live state	Peak live state per clone	Typical clone lifetime	Memory $\times$ concurrent clones
S3 GET	25 KB	2-10 KB (object metadata)	35 KB	~50 ms	1000×35 KB = 35 MB
LDAP search	30 KB	5-50 KB (result set)	80 KB	~10 ms	500×80 KB = 40 MB
DHCP request	20 KB	2 KB (lease facts)	22 KB	~10 ms	100×22 KB = 2.2 MB
Kafka consumer	30 KB	10-100 KB (batch buffer)	130 KB	~hours	100×130 KB = 13 MB
CalDAV request	35 KB	5-30 KB (event data)	65 KB	~50 ms	200×65 KB = 13 MB
Prometheus scrape	25 KB	10-50 KB (metric collection)	75 KB	~5 seconds	50×75 KB = 3.75 MB

Appendix Y — Conventional Server Software Replaced

Conventional software	Function	LM Software replacement	Development time	Key structural advantage
Nginx/Apache	HTTP server	HTTP processor runner + route rules	3-13 hours	Grammar-enforced response correctness
Postfix/Sendmail	SMTP MTA	SMTP processor runner + delivery rules	8-11 hours	Exact spam scoring with provenance
Dovecot	IMAP server	IMAP processor runner + mailbox KBs	7-10 hours	Structural user isolation
BIND/Unbound	DNS server	DNS processor runner + zone KBs	3-8 hours	Exact TTL arithmetic

Conventional software	Function	LM Software replacement	Development time	Key structural advantage
OpenSSH	SSH server	SSH processor runner + auth KBs	8-12 hours	Grant-governed command access
vsftpd/ProFTPD	FTP server	FTP processor runner + filesystem grants	5-8 hours	Grant-based path restriction
Mosquitto	MQTT broker	MQTT processor runner + topic KBs	4-6 hours	Exact QoS tracking
ejabberd	XMPP server	XMPP processor runner + roster KBs	7-10 hours	Structural roster scoping
Synapse (Matrix)	Matrix homeserver	Matrix processor runner + room KBs	12-18 hours	Structural federation safety
Redis	Cache/data store	Redis-compatible processor runner	4-6 hours	Exact sorted set scores
Kafka	Message streaming	Kafka-compatible processor runner	8-12 hours	Exact offset tracking
Keycloak	OAuth/OIDC	Auth processor runner + token KBs	9-12 hours	Monotonic grant lifecycle
OpenLDAP	Directory service	LDAP processor runner + directory KBs	7-10 hours	KB tree is native LDAP tree
Prometheus	Metrics collection	Metrics processor runner + metric KBs	3-4 hours	Exact threshold comparison
rsyslog	Log management	Syslog processor runner + log KBs	4-6 hours	Provenance per message

Conventional software	Function	LM Software replacement	Development time	Key structural advantage
MinIO	Object storage	S3-compatible processor runner	7-10 hours	Versioning as KB tree
Asterisk	VoIP PBX	SIP processor runner + call KBs	10-15 hours	Exact CDR arithmetic
HAProxy	Load balancer	Port handler with queue distribution	3-5 hours	Atomic queue pop distribution
Certbot	Certificate management	CA rules + polling runner	3-5 hours	Automatic renewal via counter expiration
SpamAssassin	Spam filtering	Prolog spam rules + VDR scoring	Included in SMTP handler	Exact scoring with per-rule provenance
fail2ban	Intrusion prevention	Rate limit counters + blocking rules	2-3 hours	Exact rate arithmetic; no false positives from float
Nagios/Zabbix	Infrastructure monitoring	Monitoring pollers + alert rules	12-18 hours (full stack)	Exact thresholds; structural alert chain
Squid	HTTP proxy	HTTP processor runner + cache LRU	5-8 hours	Exact cache TTL; grant-governed upstream

Appendix Z — Wire Format Efficiency Analysis

Protocol	Avg request size (bytes)	Grammar parse cost (tokens)	Prolog processing (tokens)	Response grammar (tokens)	Total LLM tokens	Conventional code equivalent (LOC)
HTTP GET (simple)	~200	0 (grammar)	8-16 (route + fetch)	0 (grammar)	8-16	~50-100

Protocol	Avg request size (bytes)	Grammar parse cost (tokens)	Prolog processing (tokens)	Response grammar (tokens)	Total LLM tokens	Conventional code equivalent (LOC)
HTTP POST JSON	~500	0 (grammar)	16-40 (validate + process + store)	0 (grammar)	16-40	~100-300
SMTP message	~5,000	0 (grammar)	16-30 (spam check + deliver)	0 (grammar)	16-30	~200-500
DNS A query	30-50	0 (grammar)	8 (zone lookup)	0 (grammar)	8	~50-100
MQTT PUBLISH (QoS 0)	~100	0 (grammar)	8-16 (topic route + distribute)	0 (grammar)	8-16	~30-80
IMAP FETCH	~50	0 (grammar)	8-16 (message lookup + format)	0 (grammar)	8-16	~100-200
IRC PRIVMSG	~100	0 (grammar)	8-16 (route + distribute)	0 (grammar)	8-16	~30-60
Redis GET	~30	0 (grammar)	8 (fact lookup)	0 (grammar)	8	~10-20
SSH command	~50	0 (grammar)	8-24 (parse + execute + format)	0 (grammar)	8-24	~50-150
LDAP SEARCH	~100	0 (grammar)	8-24 (tree query + filter + format)	0 (grammar)	8-24	~100-300
GraphQL query	~300	0 (grammar)	24-60 (parse + resolve tree)	0 (grammar)	24-60	~200-500

Protocol	Avg request size (bytes)	Grammar parse cost (tokens)	Prolog processing (tokens)	Response grammar (tokens)	Total LLM tokens	Conventional code equivalent (LOC)
gRPC unary	~200	0 (grammar)	16-40 (deserialize + process + serialize)	0 (grammar)	16-40	~100-300
SIP INVITE	~500	0 (grammar)	16-30 (route + state machine + forward)	0 (grammar)	16-30	~200-400
S3 GET	~200	0 (grammar)	8-16 (auth + lookup + stream)	0 (grammar)	8-16	~100-200
OAuth token	~300	0 (grammar)	16-24 (validate code + issue token)	0 (grammar)	16-24	~150-300
WebSocket frame	~50	0 (grammar)	8-16 (unmask + route + process)	0 (grammar)	8-16	~50-100
CoAP GET	~20	0 (grammar)	8 (resource lookup)	0 (grammar)	8	~30-60
AMQP basic.publish	~200	0 (grammar)	8-16 (exchange route + queue push)	0 (grammar)	8-16	~50-150
Syslog message	~200	0 (grammar)	8-16 (parse + assert + rule eval)	0 (grammar)	8-16	~30-80
NTP query	48	0 (grammar)	8 (timestamp fill)	0 (grammar)	8	~20-40

Protocol	Avg request size (bytes)	Grammar parse cost (tokens)	Prolog process- ing (tokens)	Response grammar (tokens)	Total LLM tokens	Conventional code equiva- lent (LOC)
DHCP DIS- COVER	~300	0 (gram- mar)	8-16 (pool search + offer)	0 (gram- mar)	8-16	~100-200
RADIUS Access- Request	~200	0 (gram- mar)	8-16 (auth check + grant eval)	0 (gram- mar)	8-16	~100-200
Prometheus scrape	~100	0 (gram- mar)	16-30 (collect metrics + format)	0 (gram- mar)	16-30	~50-100

[[HOWL-VDR-25-2026](#)] LLM Server Software. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-25-2026>

[[HOWL-VDR-1-2026](#)] VDR Arithmetic: Value, Decimal, Remainder. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-1-2026>

[[HOWL-VDR-2-2026](#)] VDR Gym: Exact Arithmetic Across Fifteen Domains. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-2-2026>

[[HOWL-MATH-3-2026](#)] The Transcendental Hierarchy. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH-3-2026>

[[HOWL-MATH-4-2026](#)] The Universal Power-of-Two Basis. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH-4-2026>

[[HOWL-VDR-14-2026](#)] VDR-LLM-Prolog. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-14-2026>

[[HOWL-VDR-21-2026](#)] VDR-LLM-Prolog on FPGA. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-21-2026>

[[HOWL-VDR-22-2026](#)] VDR-LLM-Prolog on Dedicated Silicon. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-22-2026>

[[HOWL-VDR-23-2026](#)] VDR-LLM-Prolog: Functional Remainder Hardware. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-23-2026>

<https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-23-2026>

[[HOWL-VDR-24-2026](#)] LLM Software. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-24-2026>

[[HOWL-VDR-12-2026](#)] Grammar-Directed Compaction and Generation. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-12-2026>

[[HOWL-VDR-5-2026](#)] Exact Arithmetic Meets Logical Provenance. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-5-2026>

[[HOWL-VDR-8-2026](#)] Computational State Primitives, Universal Data Pathing, and Session Management. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-8-2026>

[[HOWL-VDR-6-2026](#)] Computational Primitives and Operational Environments. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-6-2026>

[[HOWL-VDR-20-2026](#)] Operational Deployment. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-20-2026>